Computer Communications 154 (2020) 410–432

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Opportunistic LoRa-based gateways for delay-tolerant sensor data collection in urban settings

Nikki John B. Florita, Alyssa Nicole M. Senatin, Angela Margaret A. Zabala, Wilson M. Tan *

University of the Philippines, Diliman, Philippines

ARTICLE INFO

Keywords:
LoRa
Delay-tolerant networks
MAC protocol
Wide area networks

ABSTRACT

There are several ways that sensor nodes in a smart city setting can get data to a base station for processing. Sensor nodes that need not report their data to the base station in real time can opt for a delay-tolerant networking or data muling setup. If the data collector itself would send the data to the internet through its own (mobile) backhaul link, we can consider the data collector to be a *mobile gateway*. The choice of radio technology for the link between the sensor or end device and the mobile gateway is important: it would affect the number of mobile gateways needed, the required mobility pattern for them to provide a certain level of delay guarantees to the data, and the overall efficiency of the scheme. LoRa, with its long range and relatively low power consumption, can potentially decrease the number of gateways needed in the system, while decreasing delays. This study explores opportunistic, mobile LoRa-based gateways using a pull-based transfer of data to implement a low cost system in terms of communication and the hardware used. We present a MAC layer protocol, for use with the LoRa radio/physical layer, with three gateway operation modes: *Independent*, *Pure Pass Through*, and *Instruction-based*.

Comparison with a more traditional baseline system with push-based transfer of data reveals that our pull-based system performs better in terms of percentage of data points that are successfully sent by the end devices to the network server. We also evaluate the performance and scalability of the three modes using a small campus-wide deployment and discrete-event simulations. Experiments reveal that of the three modes, the Instruction-based mode is the most balanced in terms of computation power requirement and communication costs. In this mode, the network server handles the computation load, while the gateway handles the generation of the beacons. This shows a better result compared to the other modes wherein the computation load and beacon generation is condensed in the gateway or the network server. This also follows that the communication cost is high in the latter setup since it is dependent on the connection between the gateway and the network server. Our results also show that while utilizing multiple gateways can result in lower delays, they have to be used carefully, since the existence of multiple gateways in the system can easily result to problems with interference, possibly degrading, instead of improving, the performance of the system.

1. Introduction

A smart city utilizes the data gathered from different sensor nodes in order to make better decisions for the benefit of the city. Some situations necessitate real-time updates from the nodes, thus, the need for continuous transmission of data to a server. However, for those that do not need constant communication of the nodes to a server, such as monthly data collections for billing, the power consumption may be higher than necessary if the nodes are constantly connected compared to when nodes connect only when it is required.

A delay-tolerant network (DTN) enforces a store-and-forward transmission scheme where data from a sensor node is not immediately sent to the network server, but is collected at fixed intervals in order to reduce the energy consumption of the system. As a result, it is not necessary that the nodes are always connected to the system. This architecture may be used for smart cities in an effort to lower the power consumption. However, current implementations of DTN use protocols that operate in a short range, such as that in Firechat, which uses Bluetooth and WiFi. This implies that only a small number of sensor nodes can be accommodated in the system. This means that for the same amount of time, a device with a longer range can collect more data as it can support more nodes. The power consumption of these protocols is also dependent on the range of the devices. If the devices are placed at the farthest distance possible while still remaining in range, the power needed to correctly transmit data increases. Since the range of these protocols is short, additional power is used for data transmission.

* Corresponding author.
E-mail address: wmtan@dcs.upd.edu.ph (W.M. Tan).

https://doi.org/10.1016/j.comcom.2020.02.066
Received 19 July 2019; Received in revised form 17 February 2020; Accepted 24 February 2020
Available online 5 March 2020
0140-3664/© 2020 Elsevier B.V. All rights reserved.

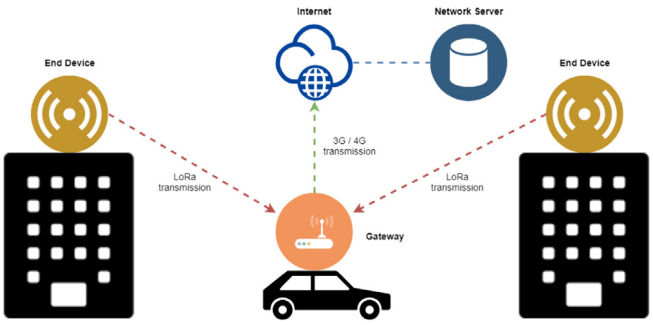


Fig. 1. Proposed LoRa system.

LoRa (Long Range) is a spread spectrum modulation technique derived from the chirp spread spectrum technology. It is a type of Low Power Wide Area Network (LPWAN) since it is capable of long range communication with a low power consumption [1].

In order to solve the range and power consumption issues mentioned, LoRa, together with DTN, can be used as a means of data transmission and collection for smart cities. Since LoRa is an LPWAN technology, it can accommodate more sensor nodes than that of short-range protocols. This implies that more data can be collected from the nodes at a certain period of time, despite having a low data transmission rate, since more nodes will be in range and can be accommodated to send their data. The use of a DTN or DTN-like setup can compensate for the low data transmission rate, since delay-tolerant data do not need to be collected right away. This also means that a mobile gateway could be used instead of several stationary gateways, since DTN allows frequent disconnections of the nodes, thus, maintaining a low cost system. Multiple mobile gateways also have the ability to circumvent the loss of communication due to obstructions, such as tall buildings and trees, and hard to reach places, such as end devices placed in subways and on top of buildings.

This paper proposes a LoRa system for collecting delay-tolerant data from stationary end devices that act as sensor nodes in an urban setting using opportunistic, mobile gateways. These gateways are mounted on vehicles that move around the city, allowing an end device to send its data only when it happens to be within the range of a gateway. This is shown in Fig. 1. Since there is a specific time wherein an end device is able to transmit the data to the gateway, as a result, the end device will save the power required to transmit data when the gateway is not in range.

A new MAC layer protocol will be created for the communication between an end device acting as a sensor node and the gateway, enabling a *pull-based transfer* of data. Different gateway operation modes will be explored, such as (1) *Independent*, wherein the gateway is in charge of sending out the beacons to pull data from the end device without the aid of the network server; (2) *Pure Pass Through*, wherein the network server will send out the beacon to the end device which will be forwarded by the gateway; and (3) *Instruction-based*, wherein the network server will give instructions to the gateway on whether it should send out a beacon to the end device.

A pull-based method of gathering data also decreases the chances of packet interference, since the end devices can only send their data when they are pulled by a gateway, compared to when the end devices can send their data freely. Even if a gateway misses the chance to pull the data from an end device, another gateway can still pull the data if it happens to pass by that end device. Since LoRa communication only happens when data is being pulled and sent, this system is not disruptive to other devices that are sharing the same frequency used by this system.

Another advantage of this system is its low cost, since mobile gateways can cover a larger area than that of stationary gateways. Multiple mobile gateways also have the ability to circumvent the loss of communication due to obstructions, such as tall buildings and trees, and hard to reach places, such as end devices placed in subways and on top of buildings.

A specific example of a system that could benefit from our proposed system is a network of temperature sensors generating data for historical analysis — the data generated need not be collected in real time (since the readings do not elicit any immediate action from authorities), and yet the data has to be collected from the sensors or end device somehow, since the end devices would not have infinite memory, and we do want the data to get to us eventually (with minimal losses). It would be costly for such a setup to have end devices capable of connecting directly to the Internet, and the cost of building stationary gateways that will cover an entire city can be expensive as well. The system will be served well with end devices capable of LoRa communication, with the data being collected by mobile gateways mounted on vehicles that more or less regularly travel across areas of the city.

The rest of this paper is structured as follows: Section 2 lists the research work related to our study, Section 3 describes the system used in our experiments, Section 4 enumerates the different modes of gateway operation, Section 5 describes the methods used in the experiment, Section 6 discusses the observed results, and Section 7 concludes the paper.

2. Related work

This section discusses the different research work involving simulations and smart city applications involving LoRa and other protocols that can be used.

A pull-based system to collect data in an urban setting and in a tree farm was utilized by Park et al. [2] using a moving gateway attached to a drone. Communication was possible at a height of 15 to 20 m (in both settings) and at a horizontal distance of 500 m (in the first setting). The results do not reflect the maximum range possible as they were limited by the batteries of the drone.

The system described in [2] is not opportunistic, since the path of the mobile gateway is predetermined during their experiment. The gateway also broadcasts beacon packets continuously which is expensive in terms of channel usage. On the other hand, our system explores the opportunistic capabilities of mobile gateways, as well as the three different gateway operation modes enabling a pull-based system.

A multi-technology opportunistic platform using both WiFi and LoRa for data gathering in smart cities using both static and moving elements was proposed by Almeida et al. [3]. The system is composed of three physical units responsible for data transfer. The data collecting units (DCU) are stations, either static or mobile, that collect sensor data. The mobile end devices serve as a relay for the DTN to deliver data to the sink stations. The sink stations are fixed network elements that are directly wired to the internet and is connected to the server. Each DCU and end device contains a WiFi manager and a LoRa manager. The system also uses a proposed MAC scheme, which is an extension of the LoRaWAN protocol.

Mobile data collection was also explored in [3] wherein environment data is collected and sent to stationary gateways using either WiFi or LoRa. This was implemented through the use of mobile end devices in contrast to ours which utilizes a mobile gateway that uses only LoRa for data collection. An architecture was also proposed wherein mobile end devices can receive and store information from other mobile end devices and forward it to a gateway. This setup has a higher power consumption, not only because it uses multiple technologies, but also in terms of radio operation when compared to our system since the mobile end devices also have to store and forward data from other mobile end devices.

A LoRaWAN module in ns-3 was implemented by Magrin et al. [4] based on the models they mentioned (Link Measurement Model and Link Performance Model). This module is also used for simulating our experiment. Using that, they tested the performance, in terms of

throughput and success probability, and the scalability of LoRaWAN. The results show a high throughput due to the orthogonality by the different spreading factors and a high packet success rate. LoRaWAN is also scalable since the addition of gateways improves the coverage and reliability of uplink communications.

A GPS-enabled mobile wireless sensor network was designed and implemented in [5]. Sensor nodes were placed on 15 livestock, sheep or goats in one of their experiments. One sheep was equipped with a GPS-enabled mobile sink node to keep track of sensor nodes that are within range. The mobile sink node collects the IDs of the sensor nodes using Zigbee and uploads them to a server along with the current GPS location of the mobile sink node. The implementation shows the potential application of a mobile wireless sensor network for monitoring livestock.

The system architecture of Choi et al. [5] is similar to our work, where a dedicated collector obtains data from end devices and forwards them to the network server. The application however, is for rural areas as it is used to monitor livestock. Our work aims to collect data from end devices in an urban setting using a gateway mounted on a car. The condition of the testing area is a great factor, since obstacles such as buildings, could interfere with the radio communication of the devices. They also used Zigbee, which could reach up to 1 km in the sub-GHz channels. However, this is expected to also deteriorate when deployed in an urban area as obstacles present in the city could interfere. LoRa can reach up to 2 km range in urban settings [6], which is farther than the range that Zigbee operates with. This implies that LoRa is better for long-range communication while utilizing low power.

Wireless sensor networks with mobile sinks (WSN-MS) for data collection using short range radio communication was explored by Yang et al. [7]. The mobile sinks notify the sensor nodes with beacons when they are present in the vicinity. Two WSN-MS were deployed to compare the effects of ETX and Contact-Aware ETX, with 20 sensor nodes arranged in a grid with a cell size of 2 m. Two mobile sinks walked around the area for 20 min. The results show that Contact-Aware ETX sent more packets than ETX within 10 s, while also having a lower average delay.

The use of mobile sinks in [7] for data collection is also found in our proposed system, where mobile gateways are used to collect data from the end devices. Both of the mobile sinks and the gateways also send out beacons in order for the sensor nodes or the end devices to let them know that there is a collector near them. However, their implementation only uses short range radio communication, thus allowing the system to operate in a small area. Our study aims to explore data collection in a large scale using LoRa.

3. The system

3.1. Components

The typical LoRa network architecture can be seen in Fig. 2. The system has three hardware components, namely, the end device, the gateway, and the network server. The end devices collect data from the environment. They are made with a microcontroller with sensor modules for data gathering and a LoRa transmitter for communication. The gateway is designed to collect data from the end devices. The network server stores and processes the data obtained from the end device.

As proof of concept, we implemented a system that uses the typical LoRa architecture. Fig. 3 shows the interaction of the devices involved in our system. This serves as a guide for the discussion of the system.

3.1.1. End device

Fig. 4(a) shows the end device used in the experiment. An Adafruit Feather 32u4 RFM9x LoRa with 915 MHz antenna served as the end device, which contains the sensor data that are sent to the gateway. A single channel was used with a base frequency of 915 MHz, a spreading factor of 7, a bandwidth of 125 kHz, 4/5 coding rate, and a transmission power of 23 dBm. An Adafruit DS1307 Real Time Clock module is also added for timestamping.

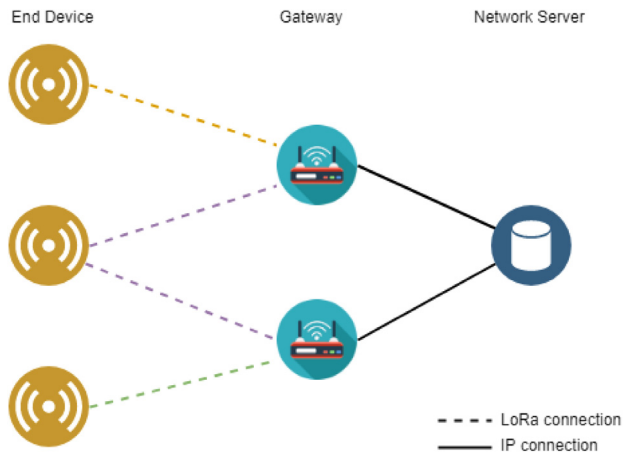


Fig. 2. LoRa architecture.

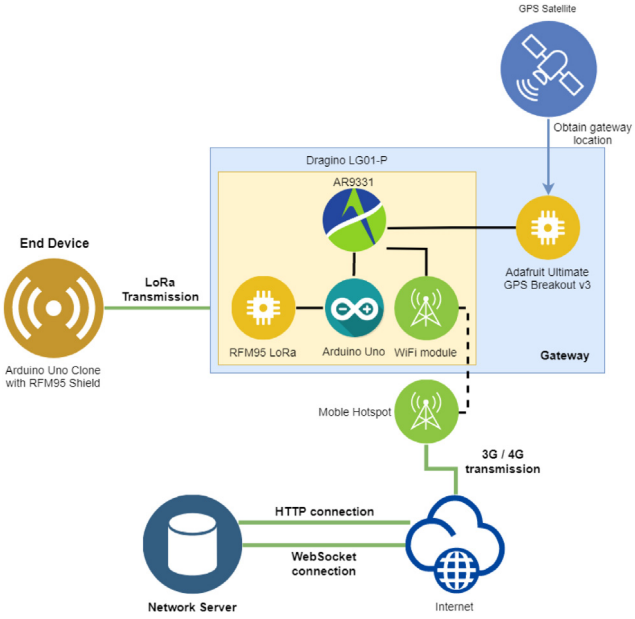


Fig. 3. Overview of the system.

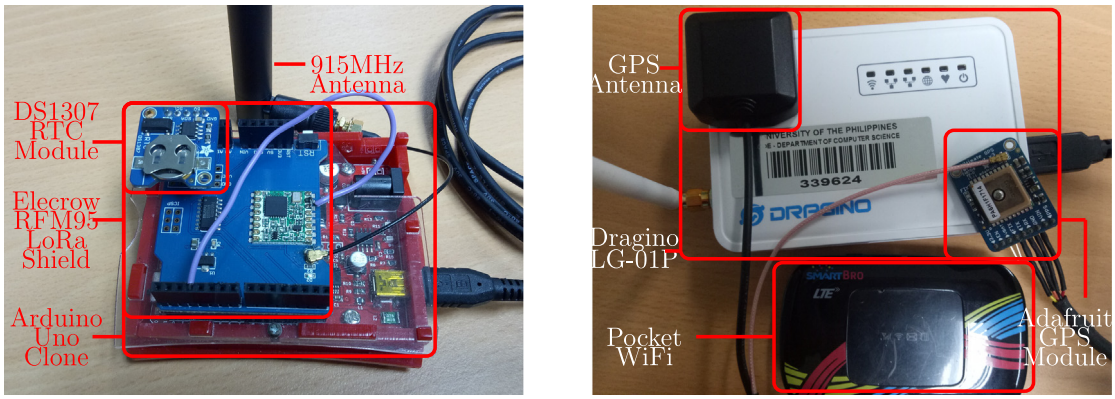
3.1.2. Gateway

The model of the gateway used in our system was Dragino LG01-P, as seen in Fig. 4(b). The Dragino gateway is composed of two microcontroller boards connected via UART, as seen in Fig. 3, one of which is a Dragino MS14-P board. The other microcontroller board is an Arduino Uno with an ATMega328P microcontroller, which is responsible for the RFM95 LoRa transmitter operation. The LoRa transmitter has the same parameters as the end device. The gateway also has a WiFi interface, and is connected wirelessly to mobile WiFi router (or pocket WiFi), which provides it with Internet connectivity. The mobile WiFi router connects to the Internet via a 3G/4G/LTE connection. An Adafruit GPS Ultimate Breakout v3 is attached to the USB port of the gateway to act as a GPS tracker.

The gateway requires a 12 V power source. For this, we used a car charger which produces 12 V and is plugged into the car's cigarette lighter receptacle. While the gateway itself is inside the car, its antenna is mounted at the roof of the car using a 2 m coaxial cable.

3.1.3. Network server

The network server is constructed using Node.js 10.15.2 with a MySQL database. The server has an IP address that is visible on the



(a) End Device (b) Gateway

Fig. 4. Hardware components.

internet. It opens a WebSocket connection on port 3000, as well as an HTTP connection on port 5000, which can also be seen in Fig. 3.

3.2. Packets

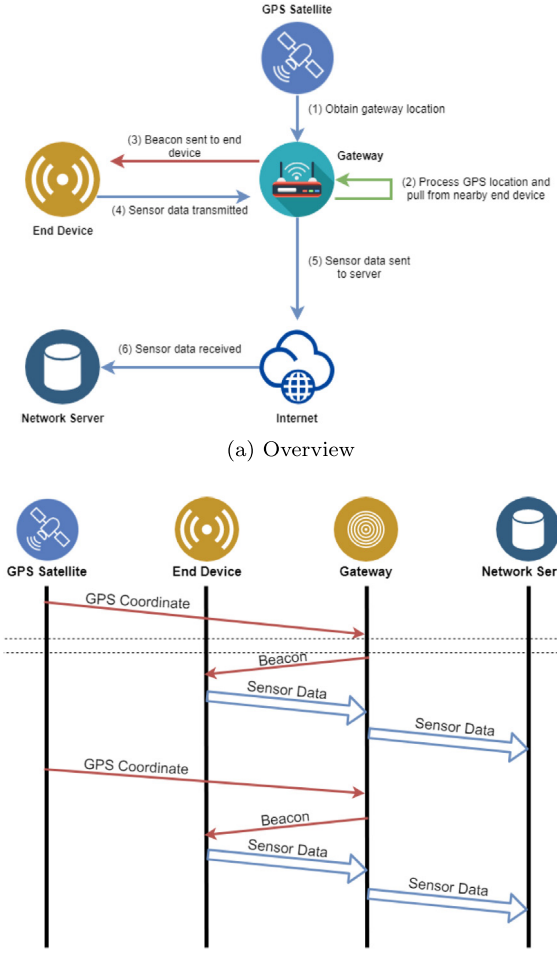
The different types of packets transmitted in the system are beacon packets, data packets, location packets, and instruction packets.

- Beacon Packet
  Beacon packets are sent by the gateway or the network server to the end devices to signal them to send their data.
- Data Packet
  Data packets are sent by the end device to the gateway and contains the data points generated by the end device throughout the experiment. These data points are differentiated by sequence numbers.
- Location Packet
  Location packets are sent by the gateway to the network server that contain the present GPS location of the gateway.
- Instruction Packet
  Instruction packets are sent by the network server to the gateway using the WebSocket connection in order to determine which end devices the gateway should send beacons to.

4. Overview of gateway operation modes

In order to minimize the power consumption by the end device and limit its usage of the radio frequency, a pull-based system is needed to facilitate the transfer of data between the devices. For our study, we explore three modes of gateway operation using a pull-based system. These are the *Independent*, *Pure Pass Through*, and *Instruction-based* modes.

The identities or IDs of end devices that must be contacted by the gateway (in response to a new GPS location reading) are placed in a *polling list*. In all the three modes, the polling lists generated by either the gateway or the network server are sorted by their distance to the gateway in increasing order. These are computed by using the Haversine function. As a result, the end devices nearest to the gateway at the current location are contacted first. As a result of using this method, the network server does not consider the frequency of data collection from an end device. Thus, using multiple gateways in the same location would not affect the number of unique end devices from which the data would be collected, since the gateways would end up computing the same polling lists.



(a) Overview

(b) Packet Exchange Diagram

Fig. 5. Independent operation mode.

4.1. Independent

Fig. 5(a) shows the flow of data in the Independent mode of operation, wherein the gateway does all of the computations. The current location of the gateway is computed by the GPS module from transmissions received from three or more GPS satellites (1). The gateway has a list of end devices and their GPS locations stored in memory, which

is filled up manually by the user. Based on the current location of the gateway, it computes its distance to every end device on the list (2). If an end device is known to be within range, the gateway will send a beacon addressed to it (3). The end device will send a data packet containing a generated data point upon reception of the beacon (4), which will be forwarded by the gateway to the network server (5, 6).

Fig. 5(b) shows the packet exchange of the system in this mode of operation. From transmissions by GPS satellites, the gateway determines its current location, and determines the end devices within range. A beacon is generated by the gateway and is sent to an end device. Upon reception of the beacon, the end device sends a data packet to the gateway, then the gateway forwards it to the network server. The encircled numbers on the right side of the figure along with the dashed lines will be discussed in Section 5.

In the Independent mode, the polling list is treated *exhaustively*. If a new GPS location is obtained and the gateway is not yet done sending beacons to the end devices in the list, the new GPS location (and its resulting new polling list) will effectively be ignored until the all the beacons for the current polling list are sent.

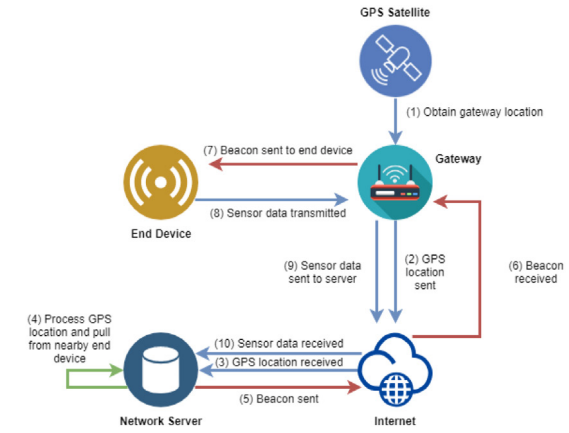
This mode of operation concentrates the computation load of the system on the gateway, which may increase the time it takes to process the beacons to be sent to the end devices. If the vehicle in which the gateway is mounted moves at a high speed, it may not be inside the range of the end device anymore and the beacon may not be sent in time. The rate of collecting data can also decrease due to the increased processing time. If the rate of collecting data cannot catch up to the rate at which an end device generates data, the data may be lost especially when the memory of the end device is small. However, this mode is resilient to disconnections between the gateway and the internet because the gateway is not dependent on the network server, as the only purpose of this link is to forward the data received from the end device.

4.2. Pure pass through

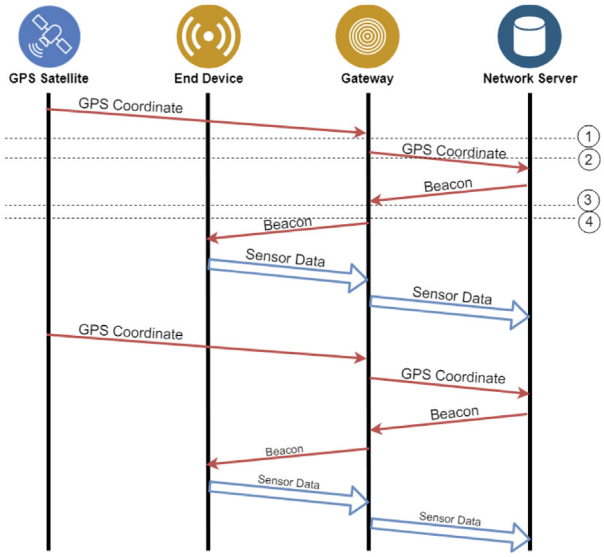
Fig. 6(a) displays the flow of data in the Pure Pass Through mode of operation, wherein all of the computations are performed by the network server. The current location of the gateway is computed by the GPS module from transmissions received from three or more GPS satellites (1). The gateway periodically sends its GPS location to the network server (2, 3). The network server has a list of end devices and their GPS locations, which is filled up manually by the user. Based on the current location of the gateway, the network server computes its distance to every end device on the list (4). If an end device is known to be within range, the network server will send a beacon addressed to it to the gateway (5, 6), which will be forwarded to the end device (7). The end device will send a data packet containing a generated data point upon reception of the beacon (8), which will be forwarded by the gateway to the network server (9, 10).

Fig. 6(b) displays the packet exchange of the system in this mode of operation. From transmissions by GPS satellites, the gateway determines its current location, and this location is forwarded to the network server. Beacons are generated by the network server and are sent to the gateway in order to be forwarded to the respective end devices. Upon reception of the beacon, the end device sends a data packet to the gateway, then the gateway forwards it to the network server. Similar to the packet exchange diagram for the Independent mode, the encircled numbers with the dashed lines will be discussed in Section 5.

In contrast to the Independent mode of operation, the gateway does not do any computations related to the pulling of data since the computation load of the system is concentrated on the network server. No processing delay will be experienced in the gateway as it only forwards the packets from the network server to the end devices, and vice versa. However, even though computations by the network server are faster compared to when they are done by the gateway, there is still a delay since the beacon will travel through the link between the



(a) Overview



(b) Packet Exchange Diagram

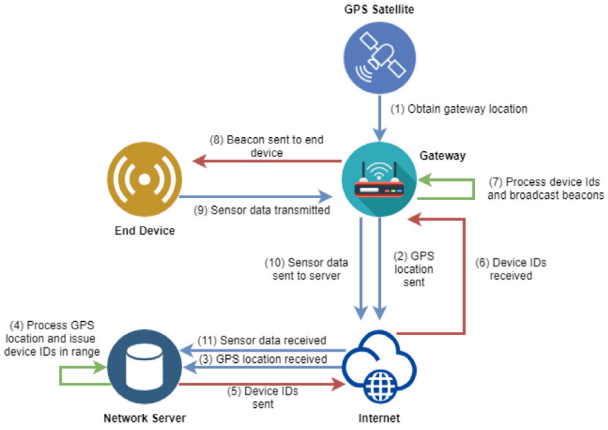
Fig. 6. Pure pass through operation mode.

network server and the gateway before being sent to the end device. This implies that the system is heavily dependent on the connection between the gateway and the internet. If a disconnection occurs, the gateway would not be able to send its location to the network server, and it also would not be able to receive the beacons sent by the network server.

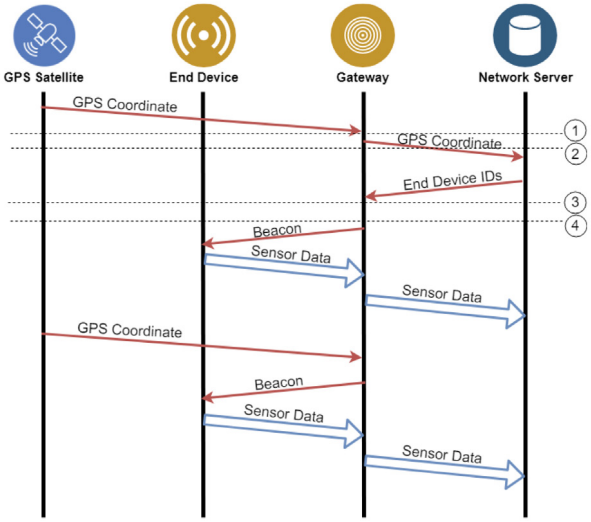
Another difference of the Pure Pass Through mode with the Independent mode is its treatment of its polling list — while the Independent mode is exhaustive in its treatment of its polling list, changing it only after all the beacons for an old one are sent, the Pure Pass Through mode readily changes its polling list as soon as a new one is computed (which happens whenever a new GPS location is acquired), even if not all end devices in the previous list had been contacted.

4.3. Instruction-based

Fig. 7(a) shows the flow of data in the Instruction-based mode of operation, wherein the computations are performed by both the network server and the gateway. The current location of the gateway is computed by the GPS module from transmissions received from three or more GPS satellites (1). The gateway periodically sends its GPS location to the network server (2, 3). The network server has a list of end devices and their GPS locations, which is filled up manually by the user. Based



(a) Overview



(b) Packet Exchange Diagram

Fig. 7. Instruction-based operation mode.

on the current location of the gateway, the network server computes its distance to every end device on the list (4). An instruction is sent by the network server to the gateway containing the list of end devices whose data should be pulled (5, 6). The gateway processes the instruction (7), and sends beacons to each of the end devices in the list (8). The end device will send a data packet containing a generated data point upon reception of the beacon (9), which will be forwarded by the gateway to the network server (10, 11).

Fig. 7(b) shows the packet exchange of the system in this mode of operation. From transmissions by GPS satellites, the gateway determines its current location, and this location is forwarded to the network server. Instructions are generated by the network server and are sent to the gateway. Beacons are generated by the gateway upon processing the instructions, and are sent to the respective end devices. Upon the reception of the beacon, the end device sends a data packet to the gateway, then the gateway forwards it to the network server. The encircled numbers along with the dashed lines will be discussed in Section 5.

The computation load in this mode of operation is divided between the gateway and the network server. This can be useful in situations where a lot of gateways are connected to the network server since the gateways shoulder the creation of the beacon, compared to the Pure Pass Through mode where the processing of the beacons to send may be slower since the network server does all of the computations. Since the instructions are sent from the network server to the gateway, this mode does not need a stable internet connection since all the tasks that the gateway needs to do are already specified in the instructions.

As for the polling list, the Instruction-based mode processes its polling list like the Independent mode, ensuring that all end devices in the polling list are contacted first before changing the list to a new one.

Table 1 shows the summary of the system components at which each packet type is generated and its destination in each of the gateway operation modes. Data packets are generated by the end device in all three modes, and are forwarded by the gateway to the network server. In the Independent mode, the beacons are created by the gateway and sent to the end device. In the Pure Pass Through mode, the location packets are produced by the gateway to be sent to the network server. The beacons by the network server are forwarded by the gateway to the end device. In the Instruction-based mode, the location and beacon packets are made by the gateway and are sent to the network server and end device, respectively. The instruction packets, on the other hand, are created by the network server and are sent to the gateway.

5. Methodology

5.1. Metrics

In order to compare the different modes of operation, we consider the following metrics: beacon packet loss, data point loss, data packet loss, overall delay, empty data buffer instances, end devices not reached, and decision delay (with its constituents local computation time and remote time).

5.1.1. Beacon packet loss

A beacon packet loss occurs when a beacon packet broadcast by the gateway is not received by any end device. This is determined by finding each beacon packet broadcast by the gateways in the end devices' logs to check if it was received.

5.1.2. Data point loss

A loss of data point occurs when it is overwritten by a newer data point in the ring buffer since it was not sent to a gateway. Data point losses are possible because of the finite size of the ring buffer which stores the data points generated by the end device. The number of these lost data points are counted in the end device.

5.1.3. Data packet loss

A data packet is lost when an end device sends a data packet but the gateway fails to receive the packet. This is determined by finding each data packet sent by the end devices in each of the gateways' data packet logs.

5.1.4. Overall delay

The *overall delay* takes into account the delay from the generation of the data until it is received by the network server.

5.1.5. Empty data buffer instances

An empty data buffer instance happens when a beacon packet for an end device arrives but there is no data point stored in the data buffer of the end device. This causes the beacon packet sent to be wasted as no data packet is transmitted back.

Table 1
Summary of the packet types generated by system components in the gateway operation modes.

Mode of operation	Packet type	Source	Forwarded by	Destination
Independent	Location	–	–	–
	Beacon	Gateway	–	End device
	Instruction	–	–	–
	Data	End device	Gateway	Network server
Pure pass through	Location	Gateway	–	Network server
	Beacon	Network server	Gateway	End device
	Instruction	–	–	–
	Data	End device	Gateway	Network server
Instruction-based	Location	Gateway	–	Network server
	Beacon	Gateway	–	End device
	Instruction	Network server	–	Gateway
	Data	End device	Gateway	End device

5.1.6. End devices not reached

We define end devices that were not reached as end devices that never got a beacon packet from any of the gateways. This can happen due to several reasons. One possible reason is topology, where the distance between the end device and any roaming gateways are too far, resulting in the end device never being added to the polling list of the gateways. Even if the end device appears in a gateway's polling list, it is possible that the polling list would be refreshed or changed even before a beacon is sent to end device. This can happen in the Pure Pass Through mode. Finally, even if a gateway gets around to sending the beacon packet to an end device, the beacon packet may end up being destroyed due to interference.

5.1.7. Decision delay, local computation time, and remote time

Upon getting a new GPS coordinate, the gateway must determine or decide which end devices it will send beacons to. We call the delay between the reception of the GPS coordinate and the sending of the beacons *decision delay*. During the decision delay, the set of end devices within range, as well as their distances, are computed and sorted. This delay is crucial, especially since the gateway is capable of movement: if the computation takes too long, the end devices may no longer be within reach of the gateway. The entities involved in computing the set of end devices vary from one mode to the other: in the Independent mode, all of the computations are done by the gateway, while in the Pure Pass Through mode, most of the computations are done by the network server. We can thus break down decision delay into two components - *local computation time*, or the delay due to computations done locally in the gateway, and the *remote time*, which includes the times it takes to send data to and from the network server, as well as the time it takes for the network server to perform the computations.

For the Independent mode shown in Fig. 5(b), the measurement of the decision delay (and local computation time) starts at ①, wherein the GPS coordinates are obtained by the gateway, and ends after all of the device IDs of the end devices to which the gateway needs to send beacons to are written into the file read by the ATMega328, as shown in ②. Remote time does not contribute to the Independent mode's decision delay.

The measurement of the local computation time for the Pure Pass Through mode seen in Fig. 6(b) is divided into two parts. The first part starts at ① when the gateway obtains the GPS coordinates, and ends at ② which indicates the time after the location of the gateway is sent to the network server. The interval between ② and ③ is not considered as part of the local computation time because it involves the transmission of the coordinates to the network server, the network server doing the computations, and the network server finally sending the list of end devices back to the gateway. This part of the operation constitutes the remote time. The second part of the local computation time begins when the beacons are received from the network server at ③, until after the beacons are processed and placed in a buffer to be broadcast at ④.

Similar to the Pure Pass Through mode, the measurement of the local computation time of the Instruction-based mode, as can be seen in Fig. 7(b) is also composed of two parts. The first part starts when the GPS coordinates are obtained by the gateway at ①, and ends after it is sent to the network server at ②. The interval between ② and ③ comprises the remote time. The second part of the local computation time begins from after the instructions are received from the network server, as shown in ③, until they are processed in ④.

5.2. Experiments

Different experiments were conducted in order to determine the feasibility of our proposed system. Parameter testing was done to obtain the range of the gateway, and the interval at which the gateway sends out beacons. The system itself was deployed and observed in the outdoor tests. These outdoor tests were then simulated in order to compare the results from the outdoor tests to their ideal values, and to determine if our system is scalable.

5.2.1. Parameter testing

Parameter testing was conducted in order to determine the interval at which the beacons can be broadcast by the network server, as well as the maximum distance at which the end device and the gateway can still communicate. In this experiment, only LoRa transmissions are tested. The connection between the gateway and the network server, along with the processing of the end devices that are inside the range of the gateway are disabled. The Round Trip Time (RTT), which is the time elapsed between the sending of a beacon and the reception of a data packet, and the Packet Reception Ratio (PRR), which is defined as the ratio between the number of data packets received by the gateway and the number of data packets sent by the end device, is measured. The end device is stationary, and generates data every 2 s. The gateway moves away from the end device in a straight line, and stops every 100 m to broadcast 100 beacons at 10-second intervals, and is done until 1 km is reached. Fig. 8 shows the locations at which the measurements took place. The RTT and the PRR are plotted against the distance traveled by the gateway.

5.2.2. Outdoor testing

The setup of the experiment is shown in Fig. 9, wherein two end devices were deployed at different positions denoted by the red circles. Both of the end devices were placed at ground level. Each end device generates data every 20 s starting from boot up. The gateway, as mentioned earlier, was inside the car with its antenna mounted on the roof, and its movement is shown with the directional arrows. It was initially positioned at the blue circle, and started moving at an average speed of 30 kilometers per hour after receiving data from End Device 0 and successfully forwarding it to the network server. Since this experiment is conducted in a university, it is possible that the car would not be able to move at a constant speed because there were stoplights and other cars that were also driving. The interval at which the gateway checks its GPS location and sends out queued beacons, and the range of the gateway were determined in the parameter testing.



Fig. 8. Parameter testing measurement locations.
Source: screenshot taken from Google Maps

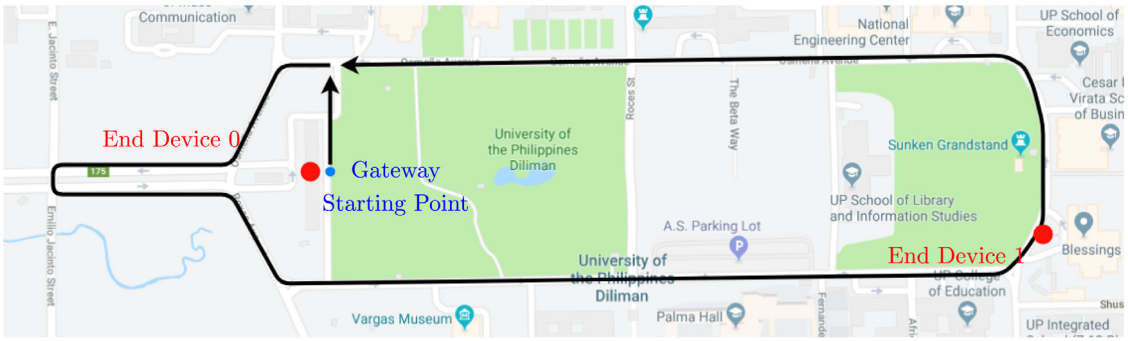


Fig. 9. Setup of the outdoor experiments.
Source: screenshot taken from Google Maps

In order to see whether multiple gateways can be used for our system, we used two gateways, each with a car, following the same setup described earlier. In this case, the second gateway starts moving 2 min after the starting time of the previous gateway. This ensures that the first gateway is already near the opposite side of the oval. Since it is possible for the two cars to catch up to each other due to stoplights, this may lead to the same results produced by the single gateway setup. In order to prevent this, the second car to arrive at the stoplight will pull over and wait for two minutes after the first car leaves before moving again.

Three runs following the two setups described were done for each mode of operation. Therefore, a total of 18 runs were done for the outdoor testing. Metrics were derived and computed based on the logs generated during the experiments.

5.2.3. Simulations

The outdoor tests were also simulated in order to compare the actual results to the ideal results. A LoRaWAN ns-3 module made by Magrin et al. [4] (updated June 9, 2018) was modified in order to take into account the different gateway operation modes described in Section 3 and the system setup described earlier. The ns-3 version used for simulation is 3.28.

A new network server class and its helper (*NewNetworkServer* and *NewNetworkServerHelp-er*) were created based from the existing network server class and its helper (*SimpleNetworkServer* and *NetworkServerHelper*). The LoRaWAN aspects contained in the latter class were removed since our experiment is only concerned with LoRa. The LoRa radio settings in the simulation were configured to correspond to the settings of the real-world radios (both end device and gateway), as stated in Section 3.1.1. In the ns-3 simulation, the *LoraChannel* class holds a list of connected PHY layers, and the relevant PHY layers are notified when a packet is sent. The relevance of a packet transmission to a PHY layer is determined by its state (whether it is idle or not), the packet receive power being above a certain threshold, and the frequency and spreading factor the receiver is listening in. After the packet is received, another check is done on whether the packet was destroyed by interference while it is being transmitted. The full link layer model is described in [4]. Packet processing functions of the network server for received packets, depending on the type of the packet received and the mode of gateway operation currently running, were added to the new network server class. When a location packet is received from a gateway, the network server computes for the end devices that are in range and sends a beacon for each end device in the case of the Independent mode, or sends an instruction packet to the gateway containing the list of end devices to send beacons to.

Applications to be installed in ns-3 nodes were also created in accordance to the functionalities of each node in the system. *EndDevice* contains a data generation mechanism that creates and stores data periodically in a buffer. Once it receives a beacon, it sends out all the data stored in the buffer and empties it. *SmartGateway*, *PassThroughGateway*, and *InstructionGateway* implements the Independent, Pure Pass Through, and Instruction-based modes of operation, respectively. *SmartGateway* periodically checks its current location, computes for the end devices that are in its range, and sends out beacons to each of the end devices. *InstructionGateway* sends its location to the network server at a specified interval. When it receives an instruction packet, it sends out a beacon for every end device on the list. *PassThroughGateway* forwards the beacons sent by the network server to the end devices.

Classes were also made in order to store the details of each end device and gateway in the system. Both *EDStatus* and *GWStatus* contain the ID of the ns-3 node these classes were attributed to, and their present locations.

In order to record the overall delay, the time by which the data is received is subtracted to the time by which the data is generated by the end device. For counting the number of packets generated by each component of the system, a counter for each packet for each component is incremented every time a type of packet is created. Since the gateway is the only component that forwards packets, it is the only one that has a counter for each type of packet that increments every time it forwards a type of packet. To account for lost data points, another counter is

also incremented every time an old data point is overwritten by a new data point. The LoRa packet loss counter is incremented whenever a callback for lost packets, which is already implemented in the module, is triggered.

To simulate the outdoor tests, a GPS trace file based on the movement of the gateways in the experiment was used to determine the movement of the gateways in simulation. The end devices were placed according to their position in the outdoor tests, and the positions were computed relative to the starting point of the gateways. To simulate the communication between the gateway and the network server, a point-to-point model was used. Since this model does not consider the network path of the packet which may affect the packet delivery time, a delay is introduced before the packet reaches the network server in order to replicate the delays occurring in the real world. This method however, always assumes that the connection between the gateway and the network server is stable and the path of the packet does not change. In effect, edge cases such as the gateway being unable to reach the cellular tower or a change in the packet's network path were not investigated. The delay between the gateway and the network server is assumed to be constant 2 ms.

The data obtained from the logs in the simulation includes the number of packets of each type generated, forwarded, and received by the different components of the system (and in the case of losses, their corresponding causes), the data points lost, and delays. This was compared to the results obtained in the outdoor tests.

The outdoor tests were also simulated with an increased number of end devices and gateways in order to determine the scalability of the system. Tests were run for 10, 50, and 100 end devices placed in random positions, and 1, 5, 10, and 20 gateways moving in the area. The metrics used to ascertain the scalability of the system will be the same as that used in the outdoor testing.

5.2.4. Decision delay, local computation time, and remote time

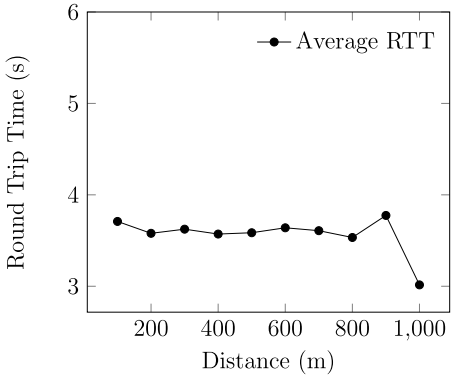
In order to measure the local computation time of the Independent mode of operation, the measurement starts with obtaining the GPS coordinates until the writing of the device IDs to the file read by the ATMega328. This routine is executed 100 times with varying numbers of end devices, starting from 100 up to 1000 in multiples of 100.

For the Pure Pass Through mode, the relevant instance for the measurement of the local computation time is when the GPS coordinate is obtained until it is sent to the network server, then the reception of the beacons from the network server until the beacons are processed and placed at the buffer. The remote time, or the time between when the GPS coordinates are sent to the network server and when beacons are received by the gateway from the network server, is also logged. This routine is also executed 100 times with the number of end devices varying from 100 up to 1000.

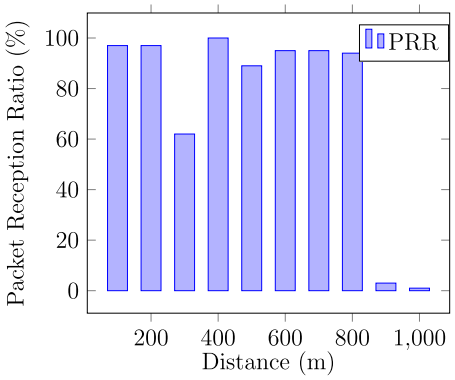
A routine in the Instruction-based mode consists of obtaining the GPS coordinates until it is sent to the network server, then the reception of the instructions from the network server until they are processed by the gateway. The remote time, or the time between when the GPS coordinates are sent to the network server and when instructions are received by the gateway from the network server, is also logged. Similar to the Pure Pass Through mode, this is also executed 100 times with the number of end devices changing from 100 up to 1000.

During this experiment, all of the components are stationary, and are within a short range from each other. To prevent other factors from interfering with the measurement of the local computation time, the GPS module is disabled and hard-coded coordinates were used. The ATMega328, which is in charge of LoRa communication, is also disabled. The end devices are listed from the farthest to the nearest in order to obtain the worst case since Insertion Sort was used to obtain the end devices nearest to the gateway in all the three modes. The timestamps at the points mentioned in the metrics section are recorded to determine the delays.

The (non-simulation) experiments conducted in this study are summarized in Table 2.



(a) Round Trip Time vs. Distance



(b) Packet Reception Ratio vs. Distance

Fig. 10. Parameter testing results.

5.2.5. Comparison with a baseline system

We also compare our proposed system to a baseline system with end devices that behave similarly to Type A LoRaWAN devices. We assume Type A LoRaWAN devices since this type is the most economical when it comes to energy usage, something of paramount importance for devices that are powered by batteries or rely on energy harvesting. A Type A device is only able to receive packets immediately after sending its own packet (during the receive window): when not sending packets or waiting for packets during the receive window, Type A device radios are assumed to be in sleep mode, during which it consumes very little energy. In contrast, the radios of the end devices of our proposed system do not go to sleep mode, since the end device must always be ready to receive beacons from the gateways. Note that Type A end devices must necessarily be *push-based*, actively reporting their data to the gateway, instead of waiting for the data to be pulled from them.

In our simulation of the baseline system, we retained the mobile aspect of the gateway. An end device repeatedly sends data packets to a gateway (at the rate of once per second), with the data packet containing the oldest data point in the circular buffer (which has 10 entries). the end device is assumed to have a data generation rate of one every 20 s. The gateway, upon receiving a data packet, acknowledges it, triggering the end device to erase the data point in its circular buffer, and to start sending the next oldest data point. Note that a data point can also be forcibly erased from the data buffer without it being sent to a gateway - this happens when 10 data points are generated without any data packet being successfully acknowledged by a gateway.

6. Results and discussion

6.1. Parameter testing

Fig. 10 shows the results of the parameter testing as a function of the distance traveled by the gateway.

Table 2
Summary of the experiments.

Experiment setup	Components used	Method	Data obtained
Single gateway	1 gateway 2 fixed end devices 1 network server	3 Rounds with independent mode 3 Rounds with pure pass through mode 3 Rounds with instruction-based mode	Overall delay Number of packets generated and forwarded Number of data points lost
Multiple gateways	2 gateways 2 fixed end devices 1 network server	3 rounds with independent mode 3 rounds with pure pass through mode 3 rounds with instruction-based mode	Overall delay Number of packets generated and forwarded Number of data points lost
Computation power	1 gateway 1 end device 1 network server	100–1000 routines in increments of 100	Execution time of program in gateway

In Fig. 10(a), the average RTT of the packet exchange between the gateway and the end device is 3 to 4 s. The average RTT is almost constant as the distance increases, but it dropped at 1km as there is only one packet that arrived.

The PRR at different distances between the gateway and the end device can be seen in Fig. 10(b). Since the experiment was conducted in an urban setting, the obstacles that affect the data transmission consist of trees, cars passing between the gateway and the end device, and people walking near the gateway and the end device. As shown in the figure, the PRR slightly dropped as the distance increased. In the case of 300 m, a waiting shed made of concrete was near the car which may have caused the PRR to drop. At distances above 800 m, only few data packets are received by the gateway which resulted to a very low PRR.

Upon analyzing the logs generated during the experiment, it was also found that at distances greater than 800 m, the beacon packets were still received by the end device, but the data packets as responses to the beacons were not received by the gateway. This may be due to the difference in size of the beacon and data packets, as the data packets are longer than the beacons by 10 bytes. The length of the packet can be a factor in the PRR of the setup.

The results of Bor et al. [6] show a 2 km effective range in an urban setting. However, it was not obtained in our results because the gateway and the end devices are all at ground level in this experiment. Results showing a range of 2 km or more were likely due to either the gateway or the end devices being placed on top of buildings which reduced the obstacles that could interfere with the communication. The use of the default parameters such as SF 7 also caused a reduction in range. Using the highest spreading factor possible as seen in their work (SF 12) would increase the effective range, but it would consume more power in order for the system to operate which would contradict the purpose of using LoRa as a low-cost and low-power method for data transmission. This is supported by Wang et al. [8] where they found that an obstructed line-of-sight can affect transmission. Similarly, Li et al. [9] suggested that obstacles have a greater influence on the PRR than the distance between the gateway and the end device.

As also mentioned by Iova et al. [10], the connectivity range is heavily affected by vegetation. It can be seen in this experiment wherein there are trees blocking the line-of-sight of the gateway when communicating with the end device. The RSSI is also affected by people blocking the line-of-sight between the devices, which is shown by Catherwood et al. [11]. Although the effect is small, people around the devices during the experiment affect the communication between the two end devices.

With these results, we decided to set the beacon interval to 5 s since there were transmissions which took more than 4 s to complete, and the range of the gateway for computing its distance to the end devices at 700 m as we expect that the effective range will be lower due to more obstacles in the outdoor testing compared to this experiment where we forced the gateway and the end device to have a relatively clearer line-of-sight.

6.2. Outdoor testing

The proposed system was deployed in a university and was tested in order to determine if it can be used as a solution for smart city scenarios. The effect of adding an additional gateway to the system was also determined.

To compare the three different modes as well as the effect of adding gateways, Fig. 11 (a–d, left bars) shows the beacon packets lost, data points lost, data packets lost, and average delays for both the single and multiple gateway experiments.

Fig. 11(a) shows the percent of beacon packets lost in each experiment setup. This is composed of beacon packets broadcast by the gateway but were not received by the end devices. The Independent mode has the least amount of beacon packets lost in the single gateway setup, followed by the Pure Pass Through mode, and finally the Instruction-based mode. All the three modes have below 2% loss which may have been caused by interference induced by obstructions such as trees, buildings, other cars, as well as the speed of the vehicle that the gateway is mounted on. This implies that comparison of the three modes in this metric is not effective due to the changing conditions of the experiment area.

In the setup with the additional gateway number of packets lost increased by more than 4% in all the three modes. This may be caused by interfering beacon packets being broadcast or the end device not in receive mode when the beacon was broadcast. Similar to the single gateway setup, the comparison of the three modes in this setup is not effective due to different experiment conditions.

The number of data points lost can be seen in Fig. 11(b). These are data generated by the end devices which were not collected by the gateway, and as a result, they were overwritten by newer data. This number does not depend on the mode since it depends on how often a gateway is near an end device to collect its data. If the gateway is rarely near an end device, the number of data points lost is expected to be large. In the single gateway experiment, the Pure Pass Through mode has the largest number of data points lost. The addition of a gateway seemingly did not translate to significant improvement for the Independent and Instruction-based mode but the number of data points lost decreased in the Pure Pass Through mode. This implies that additional gateways could decrease the data packets lost due to a gateway collecting data more frequent to a specific end device.

Fig. 11(c) shows the percent of data packets lost. Compared to the amount of beacon packets lost shown in Fig. 11(a), the percentage of data packets lost is higher. A possible reason for this is the length of packets being broadcast through LoRa. A data packet is larger compared to the beacon packet since it contains data from the end devices. The larger packet size makes the transmission more susceptible to interference. In the single gateway experiment, the Pure Pass Through mode has the highest amount of data packets lost. The amount of data packets lost increased in the multiple gateway experiment in the Independent mode and Instruction-based mode.

We note that the causes of the losses for the beacon packets and data packets can only be surmised or hypothesized at this stage, since our data source is limited to the data and packet logs of the gateways

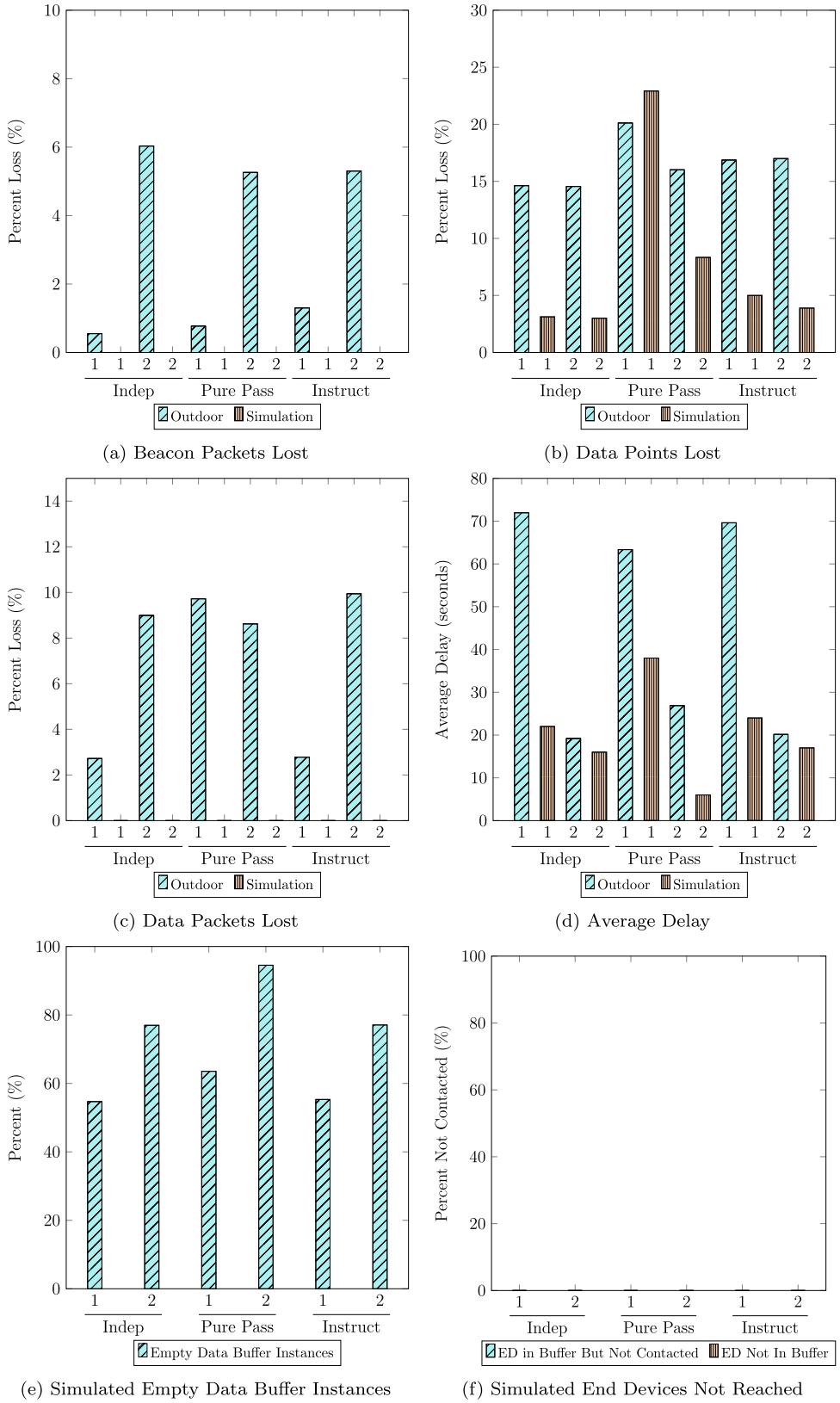


Fig. 11. Comparison of outdoor and simulated outdoor experiments.

and end devices. Future work can possibly re-run the experiments with spectrum analyzers situated near the end devices and gateways to determine with certainty the occurrence of interference.

As for the effect of utilizing multiple gateways instead of using only a single one on the average delays, the relevant experiment results are shown in Fig. 11(d). It can be seen that utilizing multiple gateways significantly improves the overall delay experienced by the data points.

6.3. Simulations - outdoor tests

The outdoor tests were simulated and the results obtained were compared with each other. Afterwards, the scalability of the system was also tested by simulating the outdoor tests with an increased number of end devices and gateways.

Note that in simulating the outdoor tests, we used asymmetric radio ranges (taking into account the values derived in Section 6.1), as well as randomized the start time of gateway's beacon sending in the setup where there are two gateways.

Fig. 11 (for subfigures a–d, right bars) shows the beacon loss, data point loss, data packet loss, average delay, empty data buffer instances, and the number of end devices not reached in the simulated outdoor experiments.

As shown in Fig. 11(a), there were no beacons lost in the simulation, while the outdoor experiment, there were some beacons lost. This may be due to the obstructions encountered by the packets in the actual experiment, while the simulation runs in an ideal environment.

The data points lost are higher in Pure Pass Through mode compared to the Independent and Instruction mode as observed in Fig. 11(b). This is possibly due to the constant buffer refresh mechanism done by Pure Pass Through, thus, favoring to contact one end device over the other. It may be seen in Fig. 11(e), that the empty data buffer instances is highest in Pure Pass Through despite it having the highest percentage of data points lost, which supports the claim that one end device is contacted more than the other one. The percentage of data packets lost in Fig. 11(c) is 0% in the simulated single and multiple gateway setup. This is different from the actual experiments, which has data packet loss of at most 10%.

The average delay in Fig. 11(d) for the simulations decreases as the number of gateways increase. This behavior is also exhibited by the actual experiments. It must be noted however that the delay magnitude for the simulated and actual experiments are not meant to be directly compared to each other because of the assumed constant network (Internet) communication delay in the simulations, as stated in Section 5.2.3.

As expected, given that there are only 2 end devices in the system, and they were deliberately placed relatively close to the road where the gateways would pass by, both were seen and contacted by the gateways (Fig. 11(f)).

6.4. Simulations - scalability testing

In order to test the scalability of the system, the outdoor tests were also simulated with an increased number of end devices and gateways. Tests were done for each mode with 1, 5, 10, and 20 gateways, with 10, 50, and 100 end devices each. Therefore, 12 setups were done for each mode of operation. The GPS traces obtained from the single gateway outdoor testing were used for the mobility of the gateway in simulation. In the case of multiple gateways, the same GPS trace was used for all the gateways, however, they start at different times. Since it takes around 500 s to traverse the path, the gateways start at a 100 s offset from the gateway before them in the 5 gateway setup in an effort to put distance between the gateways. For 10 gateways, the start time is offset by 50 s, while the offset for 20 gateways is 25 s. The end devices were positioned randomly using a random number generator. An example is shown in Fig. 12 which shows the position of the 50 end devices. This positioning is similar with the setups for 10 and 100 end devices. These were used in all of the setups in this experiment.

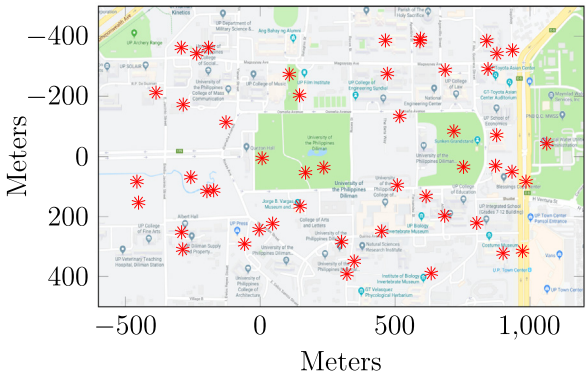


Fig. 12. Positions of the 50 end devices for the scalability testing.
Source: screenshot taken from Google Maps

6.4.1. Base parameters

We first investigate the case where the gateways send their beacons at the same time, and where the ranges of the radios (gateways and end devices) are symmetric or the same.

Figs. 13, 14, and 15 show the results of the scalability testing for 10 end devices, 50 end devices, and 100 end devices respectively.

In Figs. 13(a), 14(a), and 15(a) it can be seen that increasing the number of gateways results to a higher percentage of beacons lost. At 20 gateways, all modes, regardless of the number of end devices, registered beacon packet loss percentage greater than 90%. The losses are mainly due to interference, where packets effectively destroy each other while being transmitted, and RX conflict, where the packet arrives at the receiver's (end device's) radio, but finds it already receiving another packet. As the number of gateways increases, those destroyed due to RX conflicts increase in proportion to those destroyed due to interference. These are trends which hold true across all modes and across an increasing number of end devices.

Figs. 13(b), 14(b), and 15(b) show the percentage of data points lost as the number of gateways is increased (within each subfigure) and as the number of end devices are increased (across subfigures). Given the results concerning the beacon packets (Figs. 13(a), 14(a), and 15(a)) it is not surprising that the percentage of data points lost would increase with increasing number of gateways. Data points are lost when the buffer of the end device becomes full and data would have to be thrown away: to prevent this from happening the data must be pulled by the gateway, a process which begins with end device receiving a beacon packet. The percentage of data points lost increases as the number of end devices increases, which is to be expected, given that there would be more data points to be lost. It is interesting to note that with respect to this metric, the single gateway setup actually outperforms the multiple-gateways setups (for example, 48% for 1 gateway vs 71.6% for 5 gateways, in Independent mode, 10 end devices) - this advantage persists but rapidly diminishes at higher number of end devices however.

As for data packets lost (packets containing data, sent by end devices, but which are not received by the gateway), the values remain modest in all setups, though they increase with the number of end devices (Figs. 13(c), 14(c), and 15(c)). Given the numbers, most of the data points generated by an end device which did not make it to the network server were destroyed at the end device, before they were even transmitted.

One possible motivation for adding mobile gateways to the system is to minimize or decrease the delays between a data point's generation and its reception by the network server or database. For the Independent mode and the Instruction-based mode however, it fails to serve that purpose at all, since the single gateway setup still registered lower average delays (for example, in Fig. 13(d), 163 s for 1 gateway vs 188 s for 5 gateways, in Independent mode, 10 end devices), although

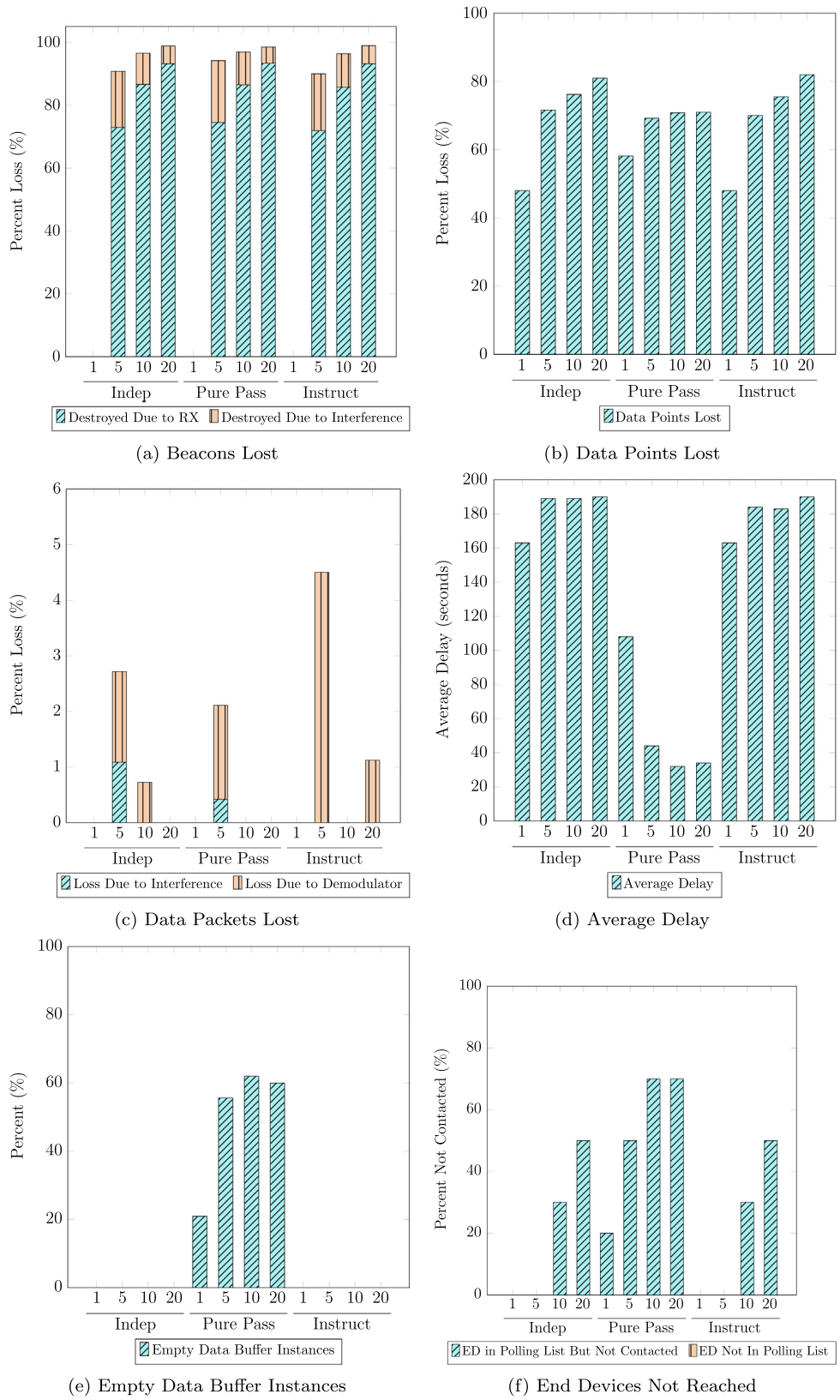


Fig. 13. 10 end devices, base parameters.

the advantage diminishes as the number of end devices increases (for example, in Fig. 15(d), 179 s for 1 gateway vs 190 s for 5 gateways, in Independent mode, 100 end devices). The reason why the single gateway outperforms the multiple gateway setups can be traced once again to the massive beacon packet losses experienced by the setups with multiple gateways.

Unlike the Independent mode and the Ins-truction-based mode, the Pure Pass Through mode setups did result in the average delay

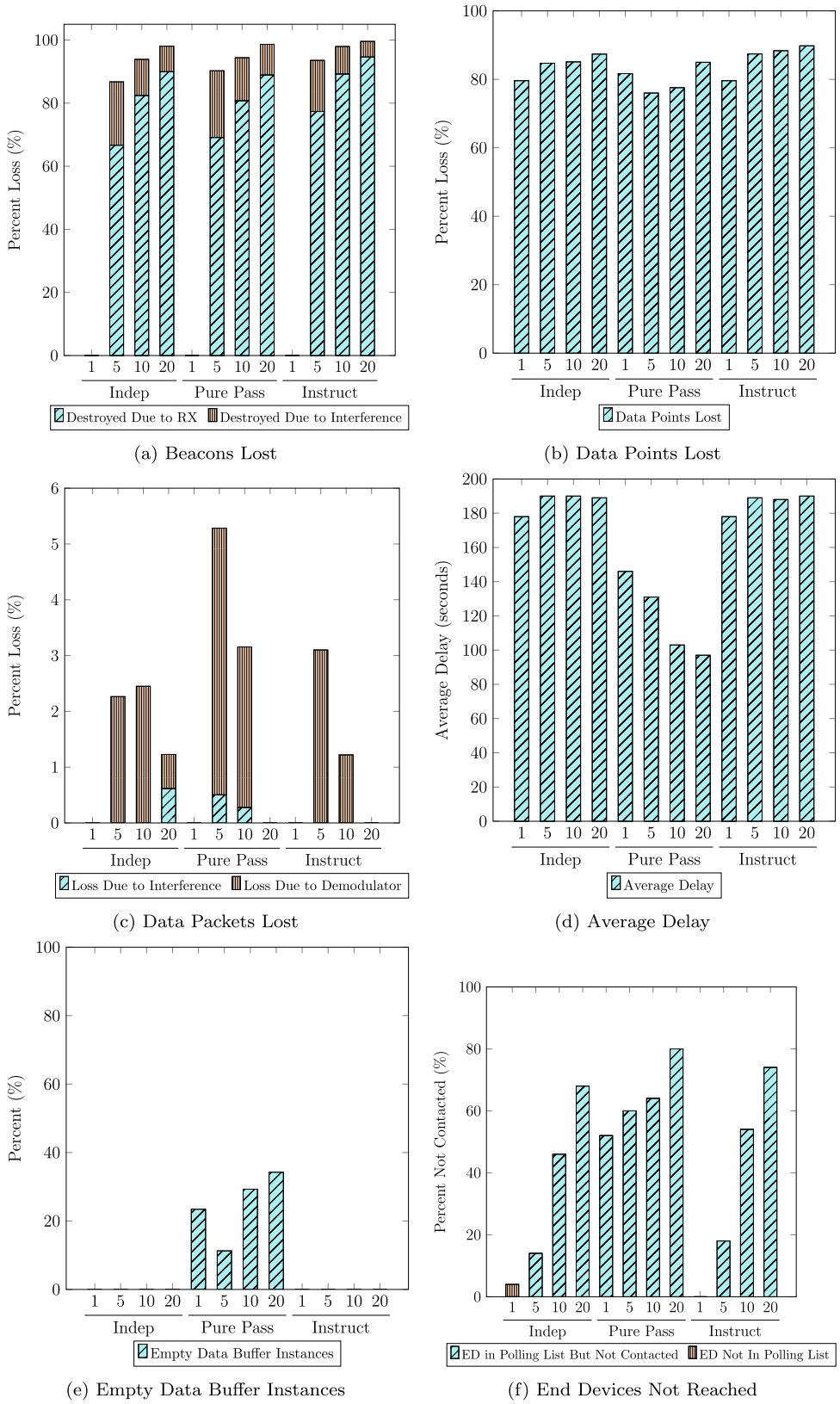


Fig. 14. 50 end devices, base parameters.

decreasing as the number of end devices are decreased (Figs. 13(d), 14(d) and 15(d)). Its advantage over the two modes decreases as the number of end devices increases, however. Investigating this trend, the performance discrepancy can be explained by the way different modes handle or refresh their polling lists (the list of end devices that a gateway will contact in response to a GPS location being obtained). The Independent mode and the Instruction-based mode are *exhaustive* in theirtreatment of the polling list: they send out all beacon packets

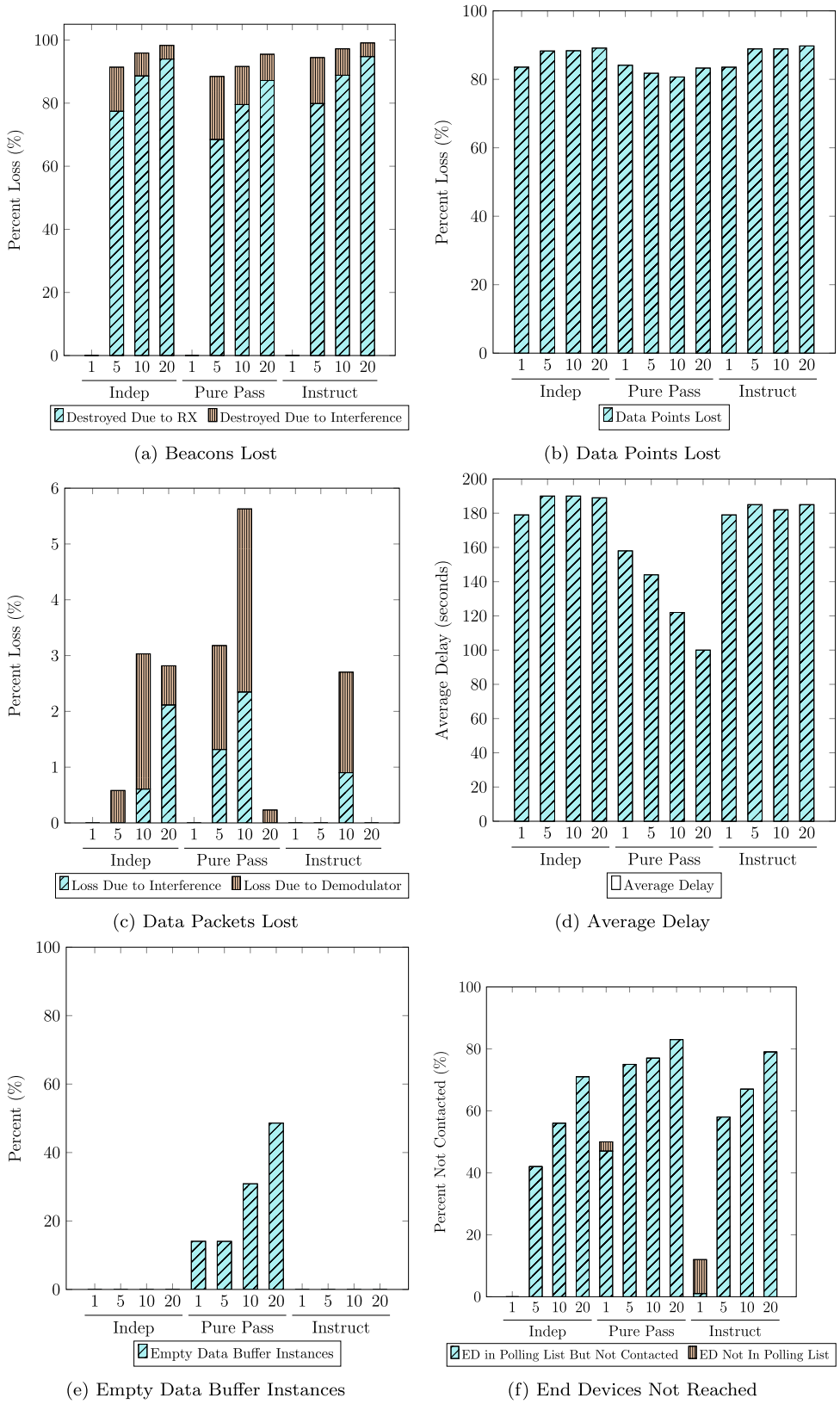


Fig. 15. 100 end devices, base parameters.

that must be sent, even if in the midst of processing, a new GPS location is acquired and a new (potential) list of end devices is produced. For example, if a GPS location results in 50 end devices populating the list (1... 50) - the Independent mode will contact (send beacons to) those 50 even before considering a new list. This has the effect of nullifying the effect of GPS locations or readings that will come in while the sending

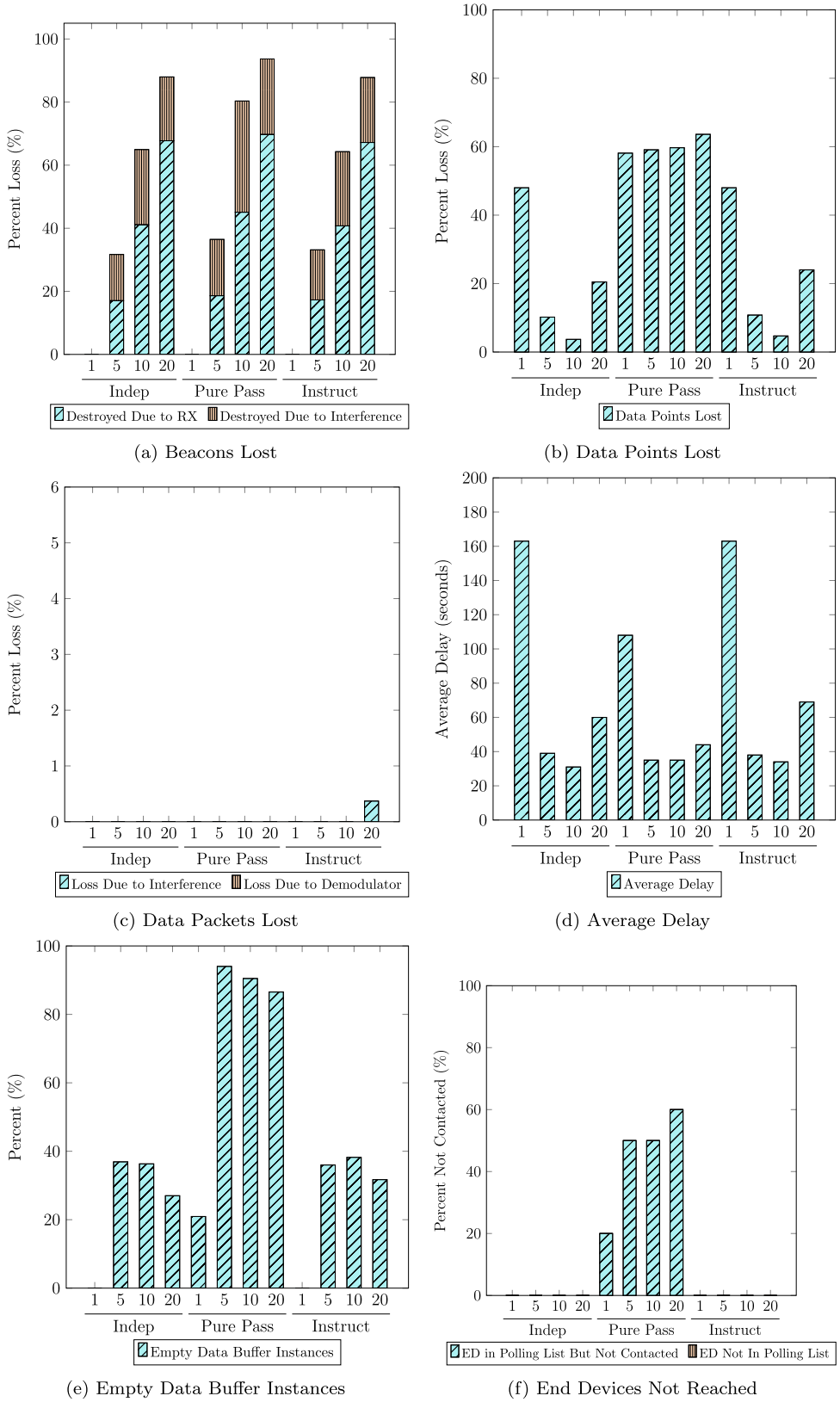


Fig. 16. 10 end devices, random beacon packet sending start time.

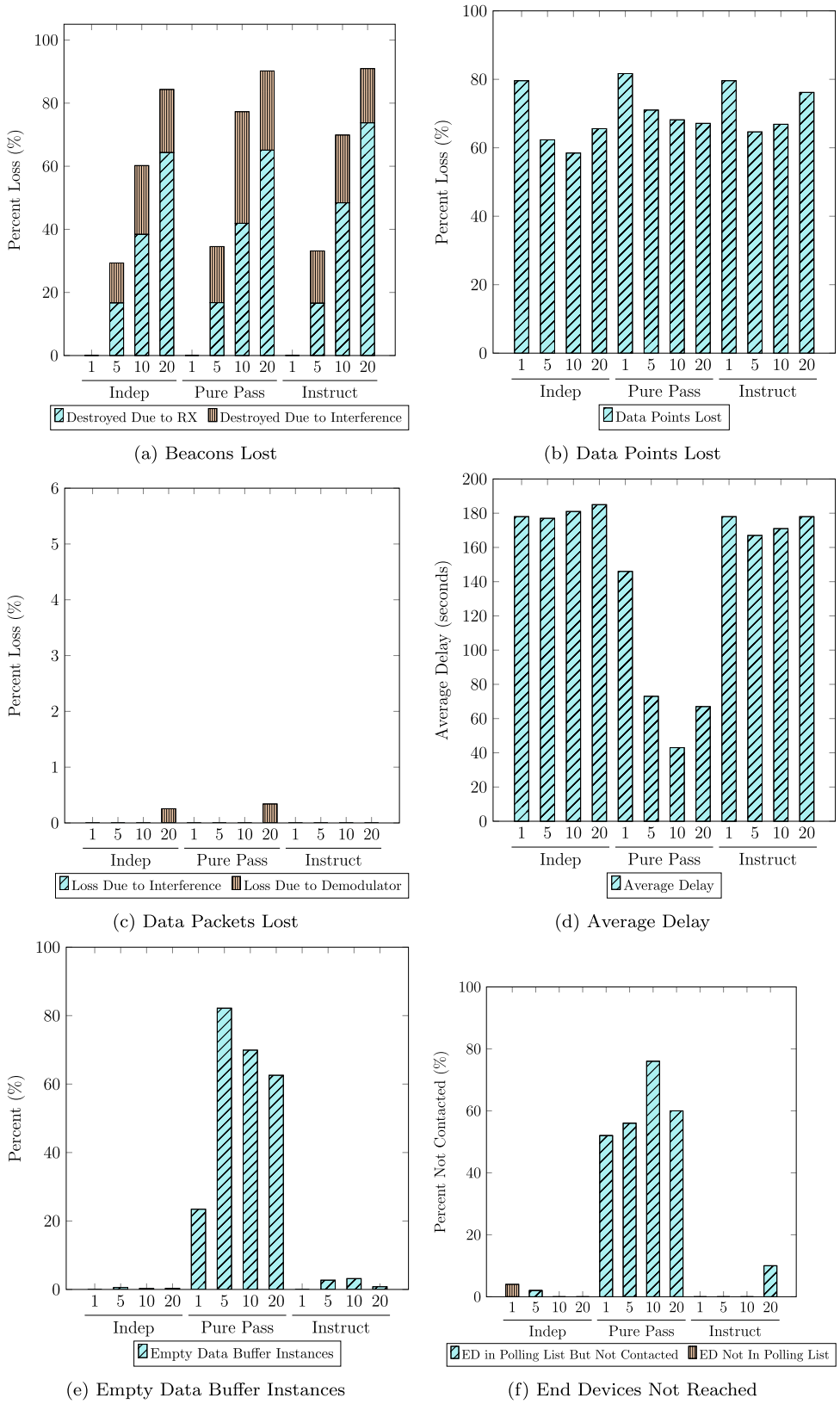


Fig. 17. 50 end devices, random beacon packet sending start time.

of beacons to the 50 end devices is not yet done. At times, gateways operating under the Independent mode or Instruction-based mode may even send beacons to end devices no longer within their range.

In contrast, the Pure Pass Through mode aggressively refreshes its polling list. Going back to the earlier example, if a GPS location results in 50 end devices populating the list (1... 50), the gateway will proceed

to contact the end devices in the list — however, once a new GPS location comes in and a new list is generated, the gateway will stop processing the old list and begin processing the new list. If the GPS location is gathered frequently and/or if a lot of end devices are in the list, those devices near the end of the list may end up not getting contacted at all.

Going back to the delay metric, the Pure Pass Through mode has low average delays, because it ends up being more *selective* or *exclusive* in the set of end devices that it attempts to contact. This is supported by Figs. 13(e), 14(e) and 15(e) - the set of devices that the gateways talk to is so exclusive that there are times (significant portions of time, actually) when the gateway contacts an end device, but the end device has no data to give yet, having already sent its data points just recently. The Pure Pass Through mode also has significantly more end devices not contacted *at all*, compared to other modes, for the same number of gateways — this can be seen in Figs. 13(f), 14(f) and 15(f)

6.4.2. Modified parameters - randomized beacon sending start time

The previous set of experiments were ran under the condition that the gateways sent their beacon packets out in unison (at the same time). The timing of the sending of the packets has a significant effect on the performance of the system, especially with regards to interference. The situation where all beacons are sent out in unison would probably be rare in real life, and thus represents something akin to a worst case scenario. To simulate a more realistic situation, we re-ran the previous set of experiments, but with the start time of each gateway's beacon sending *randomized*. We note that the gateway still sends out its beacons regularly; the only difference is that it is now not necessarily synchronized with the other gateways. The results of these experiments are shown in Figs. 16, 17, and 18.

The most drastic effect of the randomization of the beacons sending start time is in the percentage of beacon packets lost (Figs. 16(a), 17(a), and 18(a)). With the beacon sending start time randomized, the percentage of beacon packets lost is significantly lower than when the beacon packets were sent in unison (for comparison, compare Fig. 16(a) with Fig. 13(a)). The percentage of beacon packets lost still steadily increases with the number of gateways, however, which is to be expected.

With more beacon packets getting to the end devices, more data points are being sent to the gateways, resulting to less data points being lost or thrown away (Figs. 16(b), 17(b), and 18(b); compare with Figs. 13(b), 14(b), and 15(b)). More gateways generally result in lower data point losses, since there are more gateways – and beacons – to fetch the data points. However, there is an increase in data point losses at higher number of gateways such as 20, coinciding with the high percentage of beacon packets lost (which at that point is already comparable to the setups with synchronized packet sends).

The percentage of data packets lost, already relatively low in the setups with synchronized beacon packet sends, is even lower when the sending of the beacon packets are not synchronized (Figs. 16(c), 17(c), and 18(c)).

The excessive beacon packet losses in the previous set of experiments rendered the addition of gateways useless for the minimization of delays. When the sending of beacons is randomized, the addition of gateways *did* result in lower delays (Figs. 16(d), 17(d), and 18(d); compare with Figs. 13(d), 14(d), and 15(d)), although there is an increase in delay at higher number of gateways (e.g., 20 gateways), because once again, of the preponderance of high beacon packet loss percentages. Also, this effect (delay minimization) is limited by the number of end devices — the more end devices there are, the less the effects are of adding new gateways (compare Fig. 16(d) with Fig. 17(d)).

The Pure Pass Through mode is an exception to the increase in delays as the number of end devices increases (i.e., compare Fig. 16(d) with Fig. 17(d)). However, this can be explained once again by the exclusivity of the Pure Pass Through mode when it comes to the end devices that are contacted. This behavior, as we saw in the previous set of experiments, has the effect of wastefully sending beacons to end devices that have no data to send (Figs. 16(e), 17(e), and 18(e)), and ignoring or abandoning a significant portion of the end device population or set (Figs. 16(f), 17(f), and 18(f)).

6.4.3. Modified parameters - asymmetric radio ranges

One phenomenon observed in the real-world experiments is the asymmetry of the radio ranges (Section 6.1) - we incorporated that when the real-world setups were simulated (Section 6.3), but not in the setups for which we did not have real-world counterparts. To test the effect of the asymmetric radio range on the scalability of the system, we re-ran the experiment in Section 6.4.1, but with asymmetric radio ranges used. Note that the radio range is the only aspect that we changed in this set compared to the set of Section 6.4.1 – the beacons packets in this set were sent in unison, just like in Section 6.4.1.

Results (shown in Fig. 19 partially - 10 end devices only) differed minimally from those of Section 6.4.1 (Fig. 13), with the difference primarily in the number of end devices reached (see Fig. 19(f)). This indicates that the primary determining factor in Section 6.4.1's results is the synchronized sending of beacons packets, and the resulting interference, not the symmetric–asymmetric radio ranges per se. In the interest of brevity, we no longer include the full result set (50, 100 end devices) of this set of experiments.

6.5. Decision delay, local computation time, and remote time

Fig. 20 shows the comparison of the local computation time consumed by the three modes (average of 100 runs). The results in the Independent mode show that the number of end devices significantly increases the local computation time at a rate of 1.4917 s per 100 end devices. Increasing the number of end devices does not affect the Pure Pass Through mode and the Instruction-based mode, as the network server performs the computations.

Fig. 21 shows all of the factors that are involved in the decision delay for the three modes (average of 100 runs). For the Pure Pass Through and Instruction-based modes as seen in Figs. 6(b) and 7(b), respectively, the decision delay is not only composed of the local computation time, but also includes remote time.

Unlike in the Independent mode, the local computation time in the Pure Pass Through and Instruction-based modes are not affected by the increase in the number of end devices. This is because the computation in both of these modes are done on a more powerful CPU compared to that used in the Independent mode, which is the gateway. Even though both of the modes involve a remote time component, the increase in both modes is less than the increase in the gateway CPU overhead experienced in the Independent mode. Among all of the modes, the Instruction-based mode has the lowest decision delay for a given number of end devices.

Analyzing the worst case possible for all modes, which is 1000 end devices in this set of experiments, the local computation time for the Pure Pass Through and Instruction-based modes is lower than the Independent mode by more than 99.68%. However, the remote time contributes additional delays. The decision delay for the Instruction-based mode is 72.9% lower than the Pure Pass Through mode since the remote time in the Pure Pass Through mode is larger than the Instruction-based mode.

With these results, if the hardware used in future implementations is similar to the ones used in this implementation, the Instruction-based mode is the best to use as increasing the number of end devices in the system has the least effect on decision delay of this mode. However, if a more powerful gateway is used, it would have a significant effect on the Independent mode because the computation and sorting of the distances between the end devices and the gateway would be faster. Doing this for Pure Pass Through and Instruction-based modes would have little to no effect as the overhead is primarily caused by transmissions rather than the gateway CPU performing computations.

The specific local computation growth rate (Fig. 20) and local computation time — remote time ratios (Fig. 21) depicted are specific to our setup and implementation, and would highly be dependent on specific computing architectures utilized by the gateway and the network server, as well as specific network setup. Nevertheless, regardless

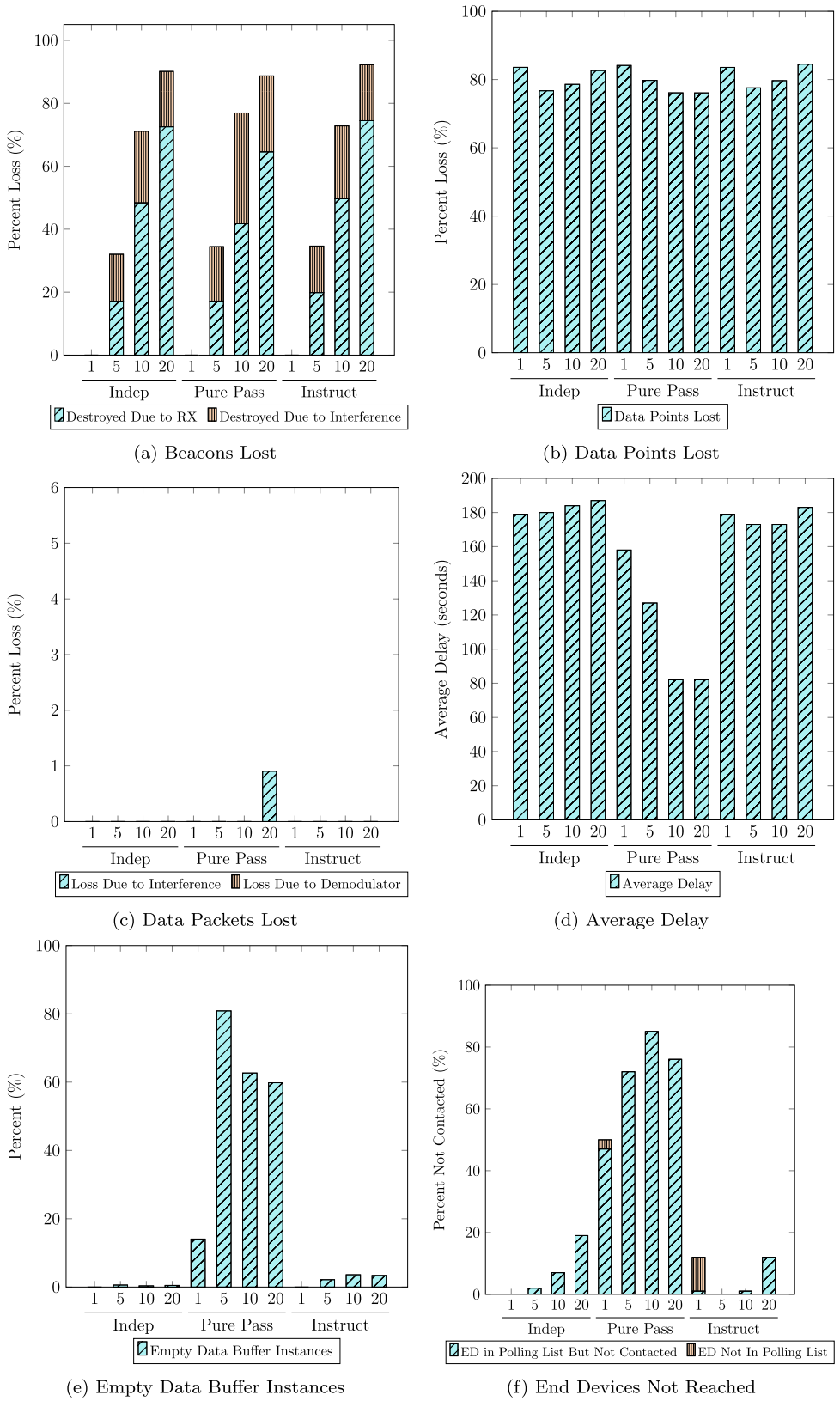


Fig. 18. 100 end devices, random beacon packet sending start time.

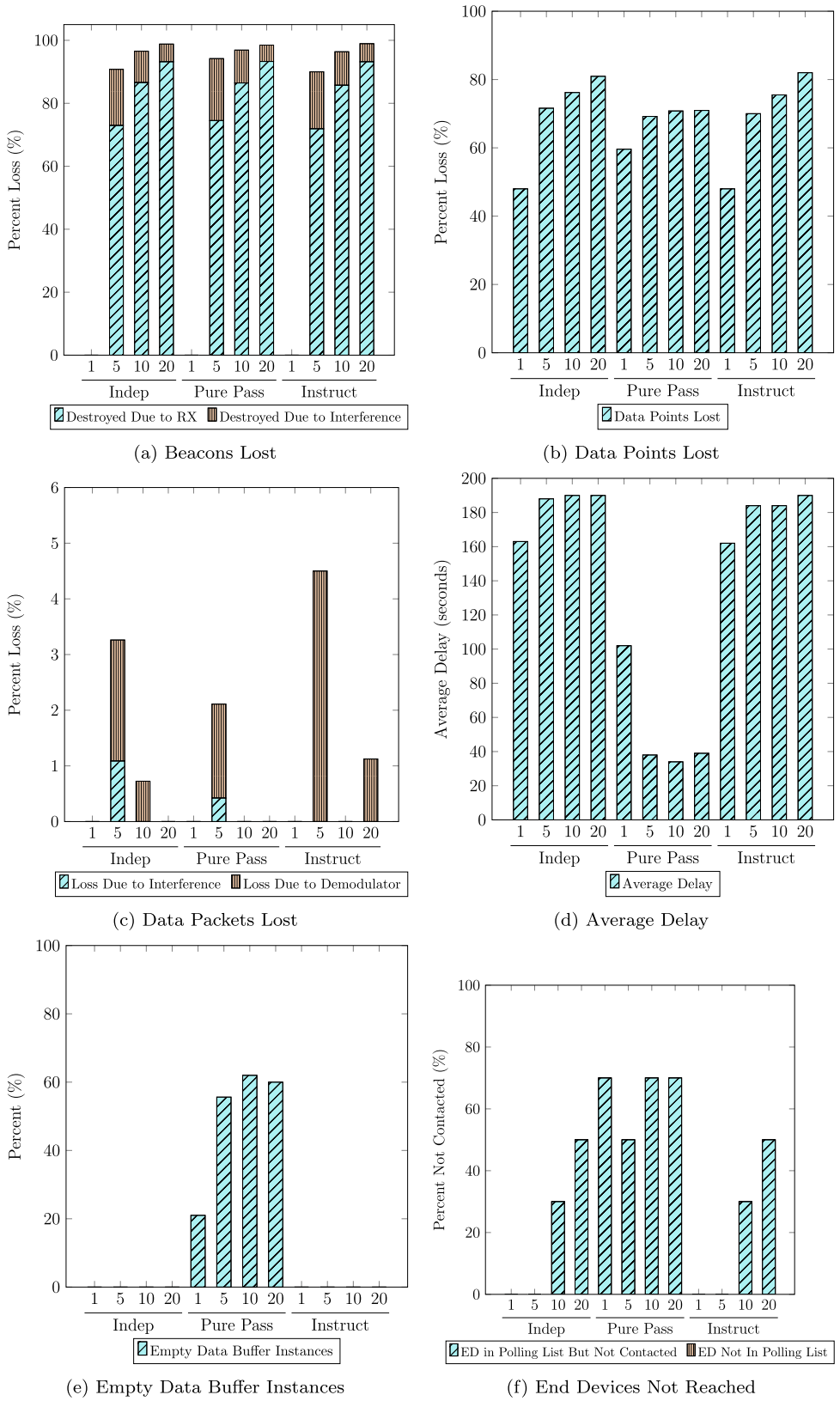


Fig. 19. 10 end devices, asymmetric radio ranges.

of computing architectures and network setup, it is likely that the Independent mode would do better in setups with computationally-powerful gateways, while the Pure Pass Through mode and Instruction-based mode would fare better in setups with more computationally-constrained gateways, assuming that the mobile Internet connection would be fast, stable, and reliable enough.

6.6. Comparison with a baseline system

The results of the baseline system experiments are shown in Fig. 22. The mobility pattern utilized, as well as send start time assumptions made are similar to the setup in Section 6.4.2, making Fig. 22 comparable to Figs. 16–18. While Figs. 16–18 have 6 subfigures each, however, Fig. 22 only has three: data points lost (Fig. 22(a)), data packets lost (Fig. 22(b)), and average delay (Fig. 22(c)). The baseline system has no beacons that facilitate the pulling of data from the end devices, hence subfigures a (beacons lost), e (empty data buffer instances), and f (end devices not reached) in Figs. 16–18 have no meaning or counterparts for the baseline setup.

Fig. 22(a) shows the percentage of data points lost in the baseline system. Data points are lost when entries in the circular buffer are erased without ever being received or successfully acknowledged by a gateway. The baseline system suffers from high data packet loss: the lowest data packet loss percentage for the baseline system is 83.19% (10 end devices, 10 gateways), which is already no longer very far from the *worst* data packet loss percentage in our proposed system (84.5%, 100 end devices, 20 gateways, Fig. 18(b)).

Fig. 22(b) provides an important clue to the trends seen in Fig. 22(a): the baseline system suffersfrom high data packet loss. With data not being sent by the end device, a significant number of buffer entries will end up being erased and discarded. The cause for the data packet losses are mainly interference as well as collisions at the receiver radio. The numbers in Figs. 16(c), 17(c), and 18(c) are significantly lower than those in Fig. 22(b), although this comparison is not exactly fair — in our proposed system, it is the beacons which are repeatedly sent and whose destruction leads to data points being destroyed (since they are not sent). Even when compared to Figs. 16(a), 17(a), and 18(a), however, the baseline system still fares worse when it comes to packet loss. This is not surprising given that in the baseline setups, where the end devices execute the repeated sends, there are more devices competing for the use of the channel than in our proposed system where the gateways (which are, in most setups, less in number than the end devices) compete for the use of the channel.

One metric where the baseline system has an advantage over our system is the average delay (Fig. 22(c)), especially at higher numbers of end devices (see Figs. 17(d) and 18(d)). This is not surprising however, given that the end devices are more aggressive in repeatedly sending their data packets than our proposed system's gateways in sending out their beacons. It must be noted however that the average delay is computed data points that are received by the network server: the data points that are not received are not taken into account. Given the results in Fig. 22(b), we can say that while a significant number of data points (and packets) do not make it to the network server, those that do so get to the network server relatively quicker than their counterparts in our proposed system.

In summary, while our proposed system suffers from interference at high numbers of gateways (leading to destroyed beacons and lost data points), the baseline system suffers the same problem at an even greater scale. This is not surprising, since in the baseline system it is the end devices that compete for the use of the channel, in contrast to our system, where the gateways do — and in general, and more realistically, there will more end devices than gateways in a system. One potential advantage of the baseline system over our proposed system is energy economy: end devices in the baseline system can go to sleep, in contrast to our system where they cannot. However, this advantage must be considered in the context of how much energy

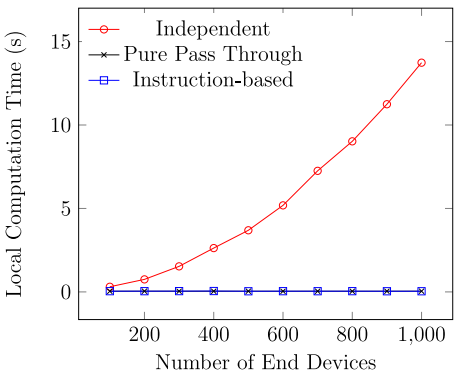


Fig. 20. Comparison of local computation time in the three modes of operation.

a send actually costs. End devices in the baseline system must do repeated sends, and sends consume significantly more energy than simply listening. While our end devices do not got to sleep since they must wait for the beacons sent by the gateways, they also do not do immensely energy-consuming sends repeatedly. The possible advantage in energy economy must also be considered in the context of the *actual performance and utility* that we get from the system in terms of the packets and data that actually get to the network server — and as our experiments show, the baseline system does not fare comparably with our proposed system in that aspect.

7. Conclusion

A smart city requires a method to collect data from nodes that are placed at different parts of the city. One such method is delay-tolerant networking or data muling, which utilizes data collectors or mobile gateways that travel around the city collecting the data and forwarding them to the internet. There are different technologies and protocols that can be used to enable the nodes to communicate with data collectors or mobile gateways: examples of these are WiFi or Bluetooth. However, LPWAN technologies such as LoRa are also very promising for use in this area, since their long communication ranges can potentially result in fewer numbers of mobile gateways needed to provide certain levels of delay guarantees.

Three different modes of operation, implementing a pull-based transfer of data using opportunistic, mobile gateways, were developed and evaluated in this study. Based on the results obtained, any of the three modes can be utilized in smart city scenarios; however, the Instruction-based mode may be the best option, since in setups that feature gateways that are significantly less computationally powerful than the network server, this mode is the fastest in terms of computation and transmission as seen in the results.

Among the three modes, investment in a more powerful gateway would only benefit the Independent mode. However, this mode is not suitable for scenarios with a large number of end devices. If a new end device is added, it would require updating the list of end devices in all of the gateways involved in the system.

Results of the scalability testing show how easy it is to get mixed results when more gateways are used in a bid to decrease the delay between a data point's generation and its reception by the network server. More gateways can result in lower delays, provided that the existence of several gateways does not result in the gateways interfering with each other's transmissions. In this aspect, the long range of LoRa turns into a double-edged sword - the same trait which makes LoRa gateways efficient even when operating in isolation causes them to decrease each other's efficacy when working together. Our experiments also show how the handling of the polling list (whether they are treated exhaustively) can profoundly affect performance, trading off delay for exclusiveness and fairness.

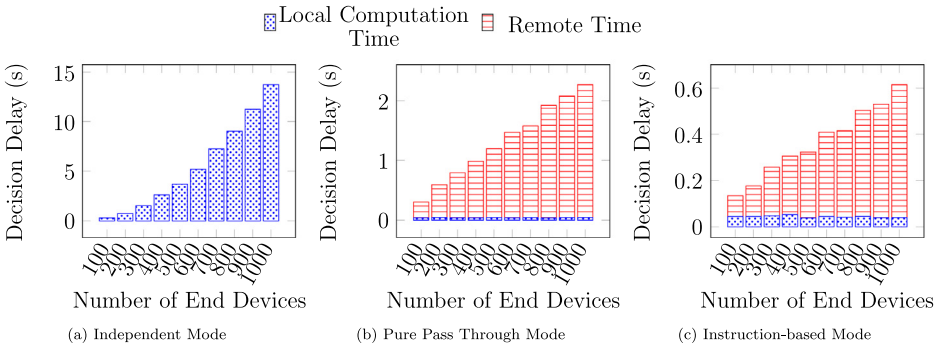


Fig. 21. Decision delay.

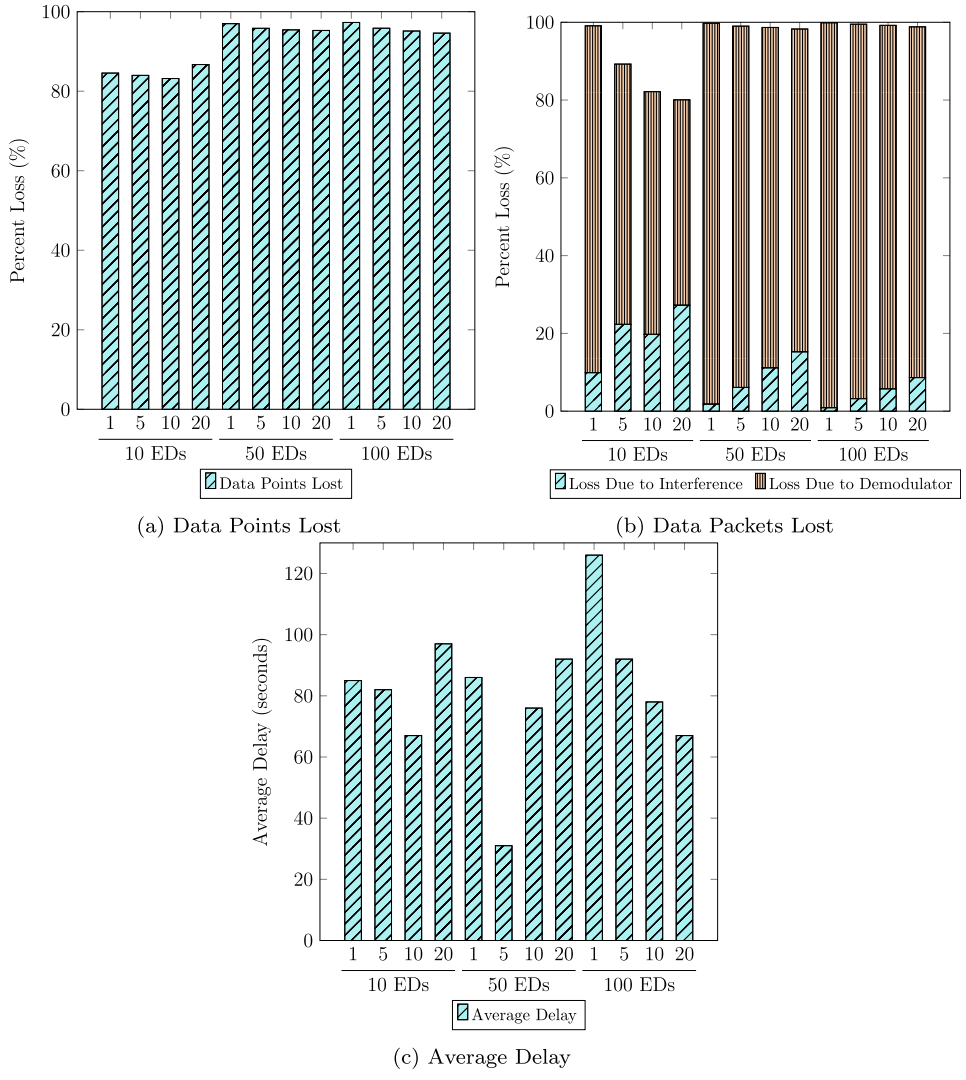


Fig. 22. Results for LoRa with LoRaWAN class a behavior simulations.

As a solution for data collection in urban settings, utilizing opportunistic LoRa-based gateways is feasible. Future research can proceed in several directions, building on the system built, tested, and characterized in this work.

For example, purely opportunistic implementations would be expensive in terms of communication between the different components involved in the system. As such, a user-triggered solution could be better, as the user could manually start data collection when an end device is in range, therefore, saving resources compared to solely relying on the GPS location to start collecting data from the end devices. Aside from that, taking note of when the end devices last sent their data and using it as an additional criteria on whether to send beacons to them or not would also improve the system. It can prioritize the end devices that were not able to send their data for a long time, compared to only getting data from the same end devices due to the high priority given to the end devices closest to the gateway. Another improvement that can be explored is greater energy efficiency. Duty cycling techniques applied to the end devices can help with this, although it remains to

be seen whether they would have a significant effect on the efficacy of the gateway's pulling of the data, given that the gateways get the data opportunistically. Future research can also explore how the mobile gateways can deal with mobile end devices. Such a situation would introduce new significant challenges, as it changes one of the fundamental assumptions of the current work: that the gateway or network server knows the locations of the end devices, and only needs to know the gateway's (or in the case of the Independent mode, its own) location to compute the list of end devices the beacons must be sent to.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Nikki John B. Florita: Software, Investigation, Writing - original draft, Writing - review & editing. **Alyssa Nicole M. Senatin:** Software, Investigation, Writing - original draft. **Angela Margaret A. Zabala:** Investigation, Writing - review & editing. **Wilson M. Tan:** Conceptualization, Methodology, Writing - review & editing, Supervision.

References

[1] M. Centenaro, L. Vangelista, A. Zanella, M. Zorzi, Long-range communications in unlicensed bands: the rising stars in the iot and smart city scenarios, IEEE Wirel. Commun. 23 (5) (2016) 60–67, http://dx.doi.org/10.1109/MWC.2016.7721743.

[2] S. Park, S. Yun, H. Kim, R. Kwon, J. Ganser, S. Anthony, Forestry monitoring system using lora and drone, in: Proceedings of the 8th International Conference on Web Intelligence, Mining and Semantics, WIMS '18, ACM, New York, NY, USA, 2018, pp. 48:1–48:8, http://dx.doi.org/10.1145/3227609.3227677.

[3] R. Almeida, R. Oliveira, D. Sousa, M. Luis, C. Senna, S. Sargento, A multi-technology opportunistic platform for environmental data gathering on smart cities, in: 2017 IEEE Globecom Workshops, GC Wkshps, 2017, pp. 1–7, http://dx.doi.org/10.1109/GLOCOMW.2017.8269110.

[4] D. Magrin, M. Centenaro, L. Vangelista, Performance evaluation of LoRa networks in a smart city scenario, in: 2017 IEEE International Conference on Communications, ICC, 2017, pp. 1–7, http://dx.doi.org/10.1109/ICC.2017.7996384.

[5] S. Choi, G. Chimeddorj, B. Altankhuyag, S. Dunkhorol, Design and implementation of a GPS-enabled mobile wireless sensor network for livestock herd tracking in mongolian nomadic herding, in: 2016 11th International Forum on Strategic Technology, IFOST, 2016, pp. 423–427, http://dx.doi.org/10.1109/IFOST.2016.7884144.

[6] M. Bor, J. Vidler, U. Roedig, Lora for the internet of things, in: Proceedings of the 2016 International Conference on Embedded Wireless Systems and Networks, EWSN '16, Junction Publishing, USA, 2016, pp. 361–366, URL http://dl.acm.org/citation.cfm?id=2893711.2893802.

[7] S. Yang, U. Adeel, Y. Tahir, J.A. McCann, Practical opportunistic data collection in wireless sensor networks with mobile sinks, IEEE Trans. Mob. Comput. 16 (5) (2017) 1420–1433, http://dx.doi.org/10.1109/TMC.2016.2595574.

[8] S. Wang, Y. Chen, T. Chen, C. Chang, Y. Cheng, C. Hsu, Y. Lin, Performance of lora-based iot applications on Campus, in: 2017 IEEE 86th Vehicular Technology Conference, VTC-Fall, 2017, pp. 1–6, http://dx.doi.org/10.1109/VTCFall.2017.8288154.

[9] L. Li, J. Ren, Q. Zhu, On the application of LoRa LPWAN technology in sailing monitoring system, in: 2017 13th Annual Conference on Wireless on-Demand Network Systems and Services, WONS, 2017, pp. 77–80, http://dx.doi.org/10.1109/WONS.2017.7888762.

[10] O. Iova, A.L. Murphy, G.P. Picco, L. Ghiro, D. Molteni, F. Ossi, F. Cagnacci, Lora from the city to the mountains: Exploration of hardware and environmental factors, in: Proceedings of the 2017 International Conference on Embedded Wireless Systems and Networks, Junction Publishing, USA, 2017, pp. 317–322, URL http://dl.acm.org/citation.cfm?id=3108009.3108091.

[11] P.A. Catherwood, S. McComb, M. Little, J.A.D. McLaughlin, Channel characterisation for wearable LoRaWAN monitors, in: Loughborough Antennas Propagation Conference, LAPC 2017, 2017, pp. 1–4, http://dx.doi.org/10.1049/cp.2017.0273.